SENTINELCHAT

Secure Multi-Client TCP Chat Platform

Final Project Report

ISEA Summer Internship 2026, Tezpur University

GIT HUB: https://github.com/Khatija-Fathima/ISEA-Phase3-TezpurUniversity-GUI-MultiClient-Chat-TCP

Document Title	SentinelChat — Final Technical Engineering Report
Document Type	Consolidated Engineering Report (supersedes Assignment 4, 6, 7, and 8 reports)
Project	SentinelChat: Secure GUI-Based Multi-Client TCP Chat Application
Prepared By	Khatija Fathima — B.Tech CSE , Andhra University
Programme	ISEA Summer Internship 2026, Tezpur University
Document Status	Final — Approved for Submission
Classification	Internal / Academic Technical Evaluation

Document Control

Revision History

Version	Date	Description	Author
A4-Draft	Jun 2026	Initial multi-client TCP prototype documented (Mininet topology, terminal client, basic broadcast).	K. Fathima
A6-Draft	11 Jul 2026	GUI layer introduced over the existing protocol; documented as a standalone assignment report.	K. Fathima
A7-Draft	16 Jul 2026	Security layer documented as a standalone assignment report.	K. Fathima
A8-Draft	16 Jul 2026	Optimization/configuration layer documented; draft reused substantial A7 text and lacked independent performance evidence.	K. Fathima
1.0 (this document)	26 Jul 2026	Consolidated final engineering report. Supersedes all prior assignment reports. Reconciles inconsistencies identified during source-code review; documents the system as a single, unified implementation.	Engineering Documentation Review

Purpose of Consolidation

SentinelChat was developed incrementally across five internship assignments (Assignment 4 through Assignment 8). Each assignment produced its own standalone report describing only the increment added at that stage. Because those reports were written independently and at different times, several inconsistencies accumulated between them — duplicated section headers, performance narratives without matching data, and, in the case of the Assignment 8 report, substantial reuse of Assignment 7 text under a new cover page.

This document replaces all four prior reports. It describes SentinelChat as it exists today — one cohesive, cumulative software system — and is sourced directly from the final codebase (server.py, the Tkinter client modules, configuration files, and accumulated log/CSV evidence), not from the earlier report narratives. Where the final implementation differs from a claim made in an earlier report, this document follows the implementation and states the discrepancy explicitly rather than silently repeating it.

Scope of This Report

This report covers the complete SentinelChat system as implemented at the time of writing: the TCP server, the Tkinter desktop client and its six feature modules, the authentication and session-security subsystem, the configuration and persistence layer, and the performance/network verification evidence collected during testing. It does not cover deployment to production infrastructure, mobile clients, or any feature not present in the uploaded source code.

Table of Contents

This section updates automatically in Microsoft Word (References → Update Table). If viewed in a reader that does not support TOC fields, see the chapter list at the end of this section.

1. Executive Summary

SentinelChat is a secure, multi-client, TCP-based chat platform implemented entirely in Python using the standard-library socket and threading modules for networking and Tkinter for the desktop client. It provides four communication modes — broadcast messaging, private one-to-one messaging, group chat, and message history review — coordinated by a single-server, thread-per-client architecture. The system was developed over five internship assignments, beginning as a bare-bones multi-client TCP demonstration and evolving into an authenticated, session-managed, configuration-driven application with an audit trail of security events and message history.

This report documents the completed system as one unified engineering artifact. It replaces every previous per-assignment report. Four engineering layers, each added in a distinct assignment but now operating as a single application, are described:

- A concurrent TCP server capable of routing broadcast, private, and group messages among multiple simultaneously connected clients.
- A Tkinter-based graphical client with a centralized message dispatcher, eliminating the need for multiple windows to independently read from a shared network socket.
- An authentication and session-security subsystem: SHA-256 password verification, duplicate-login prevention, failed-login lockout, and idle-session timeout, each backed by a persistent audit log.

The report also documents, without exaggeration, the current state of performance testing and network verification. Performance data collected to date consists of a small, single-run dataset (2-4 concurrent clients) sufficient to demonstrate the measurement pipeline but not sufficient to support strong quantitative claims; this is stated plainly in Section 11 rather than presented as a completed benchmarking exercise. Similarly, the server-status dashboard's CPU and memory indicators are simulated for demonstration purposes and are documented as such in Section 7.4 and Section 14.

Overall, SentinelChat meets its stated engineering objective: demonstrating, in a single working application, how TCP socket programming, GUI/network thread separation, application-layer authentication, and configuration-driven server design fit together in a real, testable system. Section 14 lists the specific limitations and defects identified during this review, and Section 15 proposes the engineering work required to move the system from an internship-grade demonstration to a production-grade platform (principally: transport encryption, salted credential storage, and genuine system-telemetry collection).

2. Introduction

2.1 Purpose of This Document

This document is the authoritative technical description of SentinelChat. It is written for readers who need to understand how the system is built and why specific engineering decisions were made, without first reading the source code. It is intended to support engineering review, technical evaluation, and academic assessment of the completed project.

2.2 Background

Client-server chat applications are a standard vehicle for teaching TCP socket programming because they exercise connection management, concurrency, message framing, and application-layer protocol design in a compact system. SentinelChat was developed as part of the ISEA Summer Internship 2026 at Tezpur University, beginning as a minimal multi-client broadcast server (Assignment 4) and incrementally acquiring a graphical client (Assignment 6), an authentication and session-security layer (Assignment 7), and a configuration and resource-management layer (Assignment 8). Section 3 traces this evolution in detail.

2.3 Objectives of the Completed System

- Provide reliable, ordered, multi-client communication over TCP with broadcast, private, and group messaging modes.
- Replace ad-hoc command-line interaction with a graphical client that requires no memorized command syntax.
- Authenticate users against stored credentials and protect accounts against duplicate sessions and brute-force login attempts.
- Externalize operational parameters (host, port, buffer size, timeout thresholds) into configuration rather than hardcoding them.
- Maintain a persistent, auditable record of both application messages and security events.
- Establish a repeatable process for verifying network behavior with Wireshark and measuring basic performance characteristics under increasing client load.

2.4 Scope and Non-Goals

SentinelChat is a LAN and loopback demonstration system, not a hardened production messaging platform. It intentionally does not implement transport-layer encryption, a persistent database backend, horizontal scaling, or multi-server federation. These boundaries are stated here so that the security and performance claims in later sections are read in the correct context; each non-goal that represents a meaningful limitation is revisited in Section 14.

2.5 Intended Audience

This report is written for a Chief Technology Officer or engineering review committee performing a go/no-go assessment, a security review team assessing the authentication subsystem, and an academic examiner evaluating the project against its stated internship objectives. Sections are written to be individually referenceable: a security

reviewer can read Section 6 without first reading Section 5, and an architect can read Section 4 without reading the performance appendix.

3. Project Evolution

SentinelChat was not designed top-down; it was built incrementally, with each internship assignment adding one architectural layer on top of a working system from the previous stage. This section documents that evolution so the engineering decisions in later sections can be understood in context. It is presented once, here, as a corrected and consolidated timeline — the narrative is not repeated elsewhere in this report.

3.1 Stage 1 — Baseline Multi-Client Server (Assignment 4)

The project began as a minimal multi-client TCP server tested on a Mininet topology (one server node, three client nodes). This stage established the foundational pattern retained through every later stage: a single server socket accepting connections, one worker thread per connected client, and a simple broadcast relay. Delay and throughput were measured for the first time at this stage, and Wireshark was first used to confirm the TCP three-way handshake and message delivery.

Figure 3.1 — Mininet Baseline Test Topology (Assignment 4)

Four-node Mininet topology (h1-h4) used for the original baseline connectivity and pingall verification underlying the TCP transport described in this section.

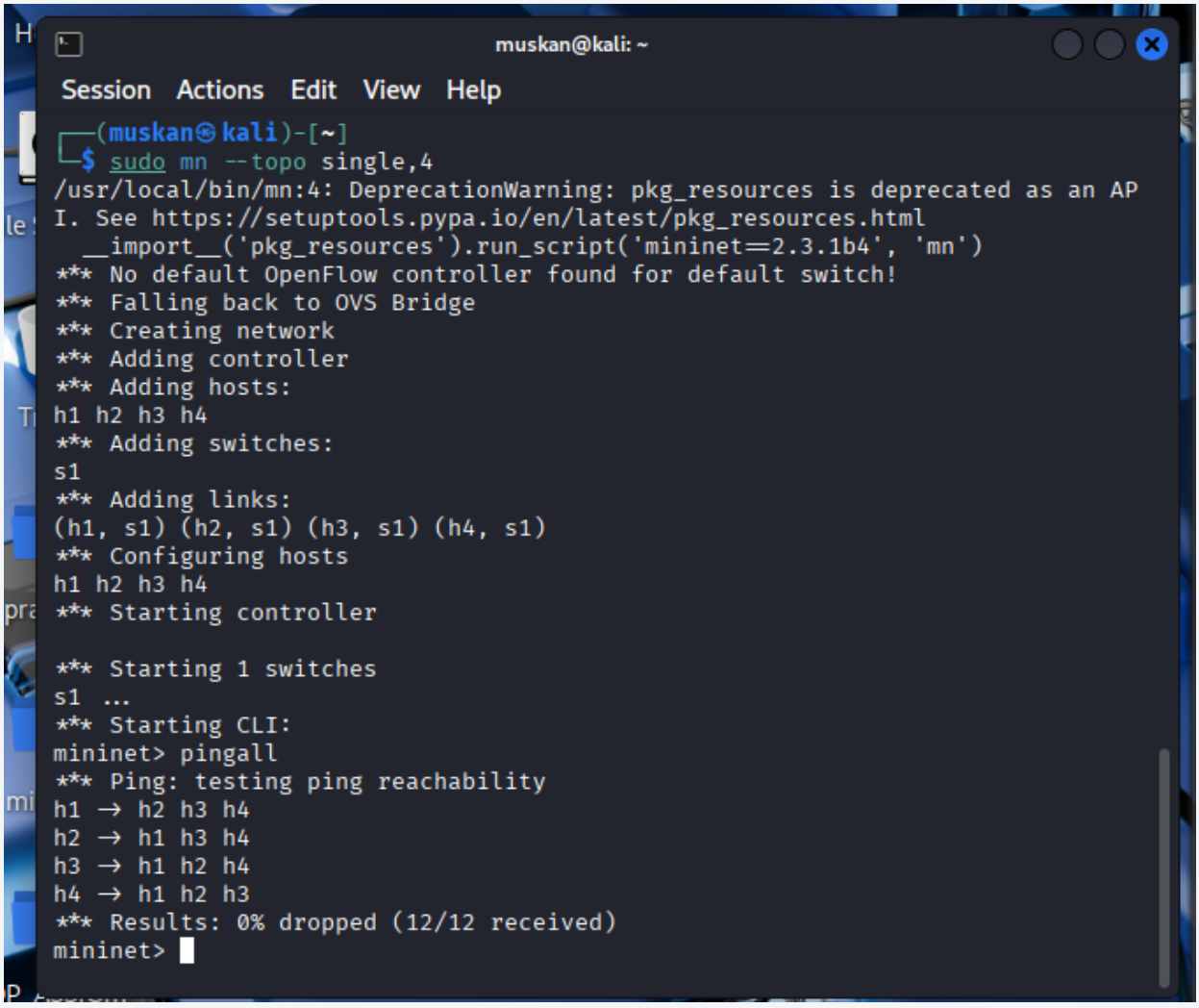

Figure 3.1 — Baseline four-node Mininet topology with full pingall connectivity.

3.2 Stage 2 — Command Protocol (Assignment 5, Inherited Baseline)

The private-message, user-listing, and group-chat command syntax (/msg, /list, /group create, /group join, /group <name> <text>, /all) that the server implements today was already in place by the time the GUI client was introduced in Assignment 6, which explicitly reused this protocol without modification. This report treats the command protocol as a stable, foundational layer documented fully in Section 5.3, rather than re-deriving it separately for each historical stage.

3.3 Stage 3 — Graphical Client (Assignment 6)

The terminal client was replaced with a Tkinter desktop application. The server was not modified. This stage introduced the architectural pattern most consequential to the rest of the project: a single dashboard window owns the client socket and the sole background receive thread, and dispatches incoming messages to whichever feature window is currently active (Section 8.2). This avoided a design where each feature window opened its own blocking recv() loop on a socket shared with other windows, which would have produced unpredictable message delivery and race conditions.

3.4 Stage 4 — Authentication and Session Security (Assignment 7)

A security layer was added on top of the existing GUI and protocol: SHA-256 password verification against a JSON credential store, duplicate-login rejection, a five-attempt lockout policy, a three-minute idle session timeout, and a persistent security event log. These mechanisms are described fully in Section 6.

3.5 Stage 5 — Configuration and Resource Management (Assignment 8)

The final stage externalized previously hardcoded operational parameters into config.json, added explicit client cleanup on disconnect (cleanup_client()), enabled SO_REUSEADDR for faster server restarts, and introduced a performance-measurement pipeline (performance_results.csv and the accompanying graph-generation script). This report treats these as genuine, verifiable improvements (Section 10) while being explicit that the accompanying performance narrative from the original Assignment 8 report was not backed by a comparable before/after dataset — see Section 11.5 for the corrected framing.

3.6 Consolidated View

The table below summarizes what each stage contributed to the system that exists today. It replaces the four separate "objective" sections that previously appeared in the individual assignment reports.

Stage	Primary Contribution	Retained In Final System
Assignment 4	Multi-client TCP server, thread-per-client model, first Wireshark verification	Yes — server concurrency model unchanged
Assignment 5	/msg, /all, /list, /group command protocol	Yes — protocol unchanged
Assignment 6	Tkinter GUI client, single-socket / single-receive-thread dispatcher	Yes — core client architecture

Assignment 7	SHA-256 auth, lockout, session timeout, security logging	Yes — active in server.py
Assignment 8	config.json, cleanup_client(), SO_REUSEADDR, performance CSV pipeline	Yes — active in server.py / graph.py

4. System Architecture

SentinelChat follows a centralized, server-mediated architecture: no client ever communicates directly with another client. Every message — broadcast, private, or group — is sent to the server and relayed by the server to its intended recipient(s). This is a deliberate simplification relative to a peer-to-peer design: it keeps the client stateless with respect to other clients (a client never needs another client's address), centralizes authentication and logging at a single point, and makes the system's behavior fully observable from one location (the server process) during testing and Wireshark capture.

4.1 High-Level Component Diagram

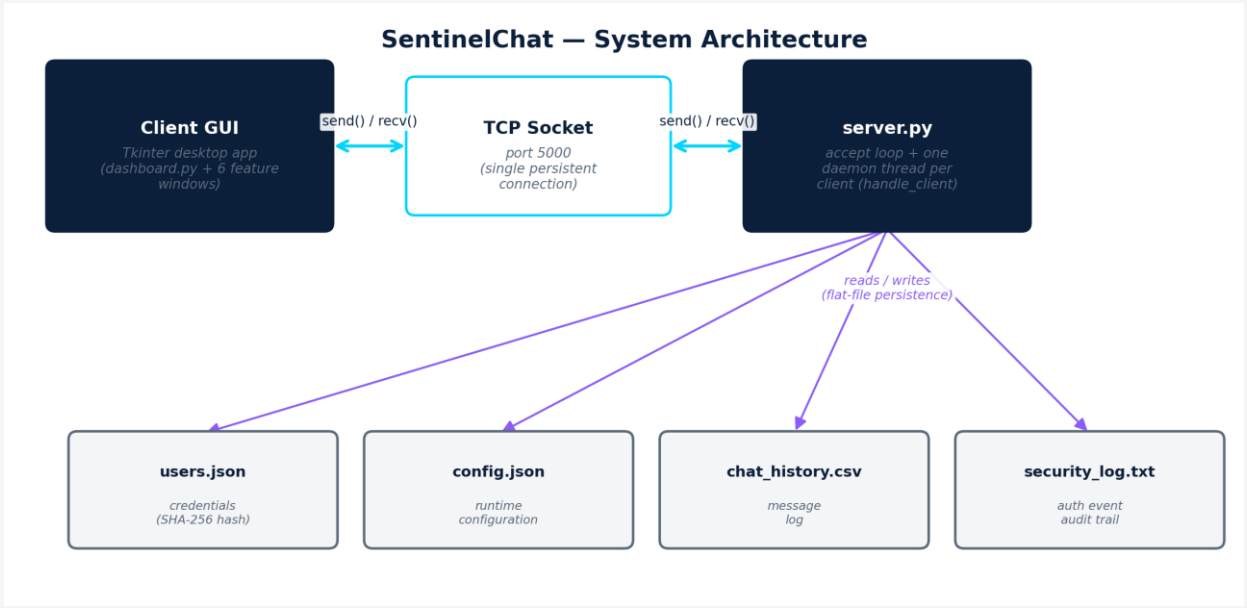

Figure 4.1 — SentinelChat System Architecture

Client GUI (Tkinter) <-> TCP Socket (port 5000) <-> server.py <-> {users.json, config.json, chat_history.csv, security_log.txt}.

Figure 4.1 — End-to-end component and data-flow diagram.

4.2 Component Inventory

Layer	Component	Responsibility
Client — Presentation	client_gui.py (login), dashboard.py, broadcast.py, private_chat.py, group_chat.py, history.py, status.py	User interface, local rendering, user input capture
Client — Support	cyber_bg.py, ui_fx.py	Shared visual-effects helpers (animated background, glow effects, chat bubble rendering) with no networking responsibility
Transport	Python socket (TCP), port 5000 by default	Reliable, ordered, connection-oriented delivery between one client and the server

Server — Core	server.py	Connection acceptance, authentication, message routing, session and lockout enforcement, logging
Persistence	users.json, config.json, chat_history.csv, security_log.txt, performance_results.csv	Durable storage for credentials, configuration, chat history, security events, and performance samples

4.3 Architectural Principles

4.3.1 Centralized Routing

All routing logic (who receives a given message) lives in server.py. Clients issue intent ("send this to user X", "send this to everyone", "send this to group Y") and never resolve recipients themselves. This keeps the client implementation simple and ensures message-history logging happens exactly once, at the point of routing, regardless of which client sent the message.

4.3.2 One Thread Per Connected Client

The server spawns a dedicated daemon thread per accepted connection (threading.Thread(target=handle_client, args=(client,), daemon=True)). This model was chosen over a single-threaded event loop because it maps directly onto the blocking socket.recv() call used throughout the implementation, keeping the per-client logic linear and easy to reason about. Its principal limitation — thread-per-connection does not scale to very large numbers of concurrent clients as well as an asynchronous I/O model would — is acceptable given the application's target scale (config.json specifies MAX_CLIENTS = 20) and is discussed further in Section 14.

4.3.3 Separation of Networking and Presentation

No Tkinter widget code exists inside server.py, and no socket.send()/recv() call exists inside a widget-construction method on the client. Networking calls are triggered from event handlers (button clicks, Enter key bindings) and their results are marshaled back onto the Tkinter main thread via root.after(0, callback, data) — never applied directly from the background receive thread. This separation is what allows the server to be tested independently with the plain terminal client (client.py) while the GUI is developed and modified without touching networking code, and is why Assignment 6 was able to introduce an entirely new client without any change to server.py.

4.3.4 Configuration Over Hardcoding

Seven operational parameters — server host, port, socket buffer size, session timeout, maximum login attempts, lockout duration, and maximum client count — are read once at server startup from config.json rather than being embedded as literals in server.py. This allows the same server binary to be redeployed to a different host or tuned for different timeout/lockout policy without a code change, and is the clearest example of the Assignment 8 configuration-management objective being realized directly in the code.

5. Communication Protocol and Networking Layer

5.1 Transport Layer

SentinelChat uses a single persistent TCP connection per client, established once at login and held open for the duration of the session. TCP was chosen over UDP because the application requires reliable, ordered delivery of both authentication handshake data and chat messages; retransmission and in-order delivery are handled transparently by the operating system's TCP stack rather than being reimplemented at the application layer.

5.2 Connection Lifecycle

- 1. Server binds to (SERVER_HOST, SERVER_PORT) from config.json and listens with a backlog of MAX_CLIENTS.
- 2. Client opens a TCP connection to the configured host and port.
- 3. Client immediately sends a pipe-delimited login packet: LOGIN|<username>|<password>.
- 4. Server authenticates synchronously (Section 6.1) and responds with one of: LOGIN_SUCCESS, LOGIN_FAILED, DUPLICATE_LOGIN, or ACCOUNT_LOCKED.
- 5. On LOGIN_SUCCESS, the server registers the client in its in-memory session tables and spawns a dedicated handler thread; the client opens the Dashboard and starts its own background receive thread.
- 6. The connection remains open until: the user disconnects deliberately, the server detects SESSION_TIMEOUT idleness, or the socket fails (network error, peer reset).
- 7. On any termination path, the server calls cleanup_client() to remove the client from every in-memory tracking structure and close the socket (Section 10.1).

5.3 Application-Layer Command Protocol

Above raw TCP, SentinelChat defines a compact, text-based command protocol. Every message the client sends after login is a UTF-8 encoded string; the server inspects a fixed prefix to decide how to route it. This is a deliberate minimalism: no binary framing, no length prefixes, and no serialization library are used, which keeps the protocol trivially inspectable in a Wireshark capture (Section 12) at the cost of not being robust against multi-packet message reassembly for very large messages — a limitation noted in Section 14.

Client Sends	Server Behavior	Delivered To
/msg <user> <text>	Looks up <user> in the active session table; forwards as [PRIVATE] <sender>: <text>; logs as PRIVATE	Named recipient only
/all <text>	Forwards as [BROADCAST] <sender>: <text> to every connected client; logs as BROADCAST	All connected clients (including sender)
/group create <name>	Registers a new empty group if the name is not already taken; logs as GROUP_CREATE	Confirmation to sender
/group join <name>	Adds the sender's socket to the named group's member list; logs as GROUP_JOIN	Confirmation to sender
/group <name> <text>	Forwards as [GROUP <name>] <sender>: <text> to every member of that group; logs as GROUP	Group members only
/list	Returns the current online-username list	Requesting client only

(any other text)	Treated as an unscoped chat message; relayed to all other clients; logs as NORMAL	All other connected clients

5.4 Message Routing Engine

Routing is implemented as a set of small, purpose-specific functions in server.py rather than one generic dispatcher: broadcast() and broadcast_all() for unscoped and /all traffic respectively, private_message() for one-to-one delivery, and group_message() for group delivery. Each function iterates its relevant recipient collection (clients, usernames, or a specific groups[name] list), attempts client.send(), and on a connection error (ConnectionResetError, BrokenPipeError, OSError) invokes cleanup_client() for the failed peer rather than allowing the exception to propagate and terminate the sender's handler thread. This defensive pattern is what allows one client's unexpected disconnection to be silently absorbed without disrupting message delivery to the remaining connected clients.

Note on implementation duplication: two broadcast-style functions exist in server.py — broadcast(), which excludes the sending socket, and broadcast_all(), which includes it and is the one actually wired to the /all command. broadcast() is not reachable from any current client command and should be treated as retained utility code rather than an active code path; this is flagged for cleanup in Section 14.

6. Authentication and Session Security

Section 4.3 established that all client intent is routed through the server; this section describes the gate every client must pass before that routing is available to them. Authentication and session security were introduced as a dedicated engineering layer (Section 3.4) specifically to prevent unauthenticated access, credential exposure, brute-force guessing, and unattended-session abuse.

6.1 Login Sequence

Authentication happens synchronously inside the server's connection-accept loop, before a per-client handler thread is spawned. The server reads the LOGIN|<username>|<password> packet, checks whether the username is currently in a temporary lockout window, and if not, calls authenticate_user(), which hashes the supplied password and compares it against the stored hash in users.json. Every outcome — success, wrong password, unknown account, active lockout, or duplicate session — is written to security_log.txt (Section 6.6) before the server responds to the client.

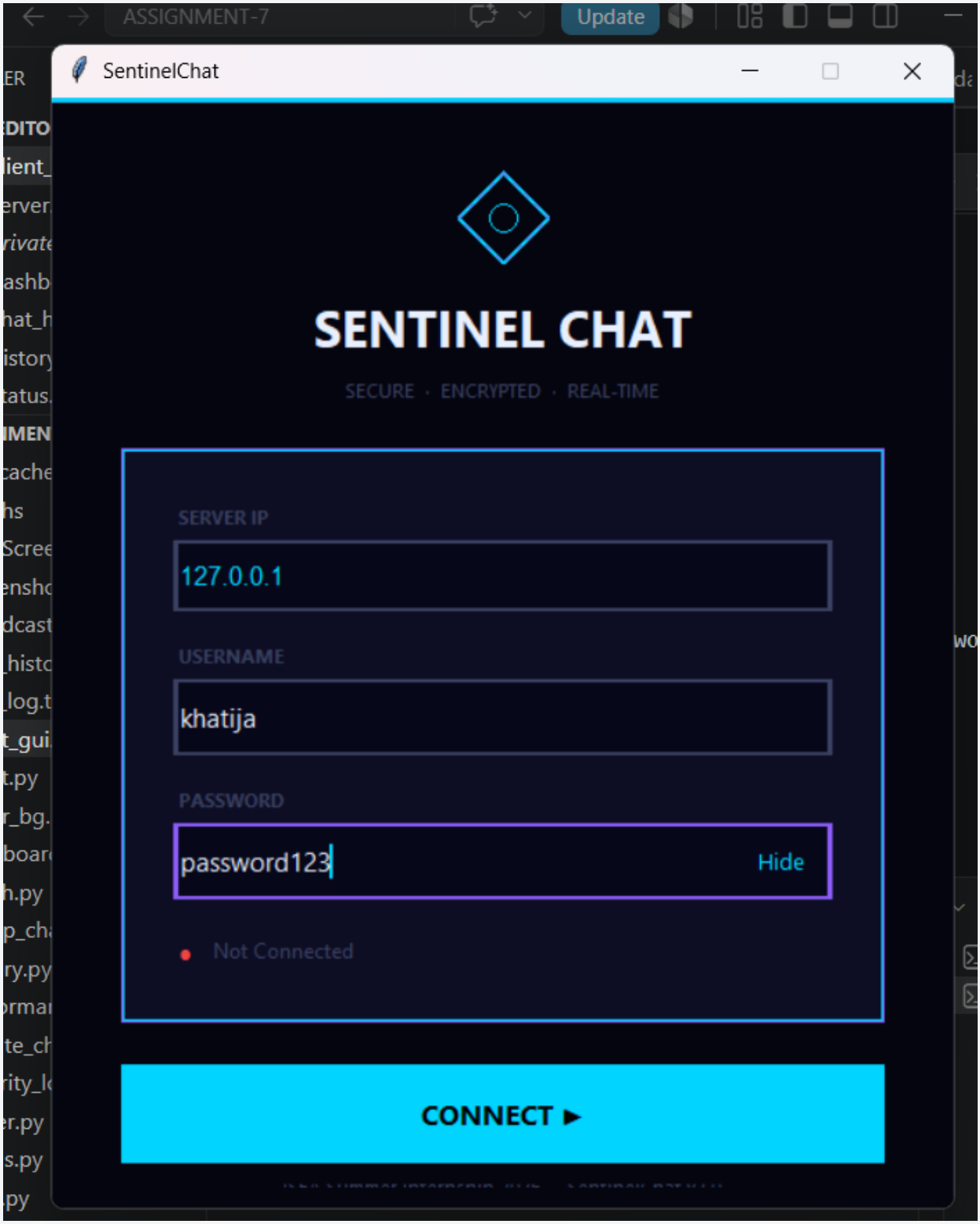

Figure 6.1 — SentinelChat Login Window

Login window showing the server IP, username, and password fields with the connection-status indicator.

Figure 6.1 — Login interface with connection status indicator.

6.2 Password Storage — SHA-256 Hashing

Passwords are never stored or transmitted-for-storage in plaintext form on disk. Each credential in users.json is a SHA-256 digest (hashlib.sha256(password.encode()).hexdigest()) of the corresponding password. At login, the server hashes the supplied password with the same function and compares digests; the plaintext password supplied by the client exists only transiently in server memory during that comparison.

Design decision and limitation: SHA-256 was chosen for simplicity and to demonstrate the principle of never persisting reversible plaintext credentials. It is a fast, general-purpose cryptographic hash, not a password-hashing function such as bcrypt, scrypt, or Argon2, and no per-user salt is applied. This means two users with identical passwords produce identical stored hashes (observably true in the current users.json, where all five demo accounts share one hash), and the stored digests remain vulnerable to precomputed dictionary/rainbow-table attack if the credential file were ever disclosed. This is documented as a known limitation in Section 14 rather than presented as production-grade credential storage.

> **ENGINEERING NOTE — Why SHA-256 Alone Is Not Enough**
>
> SHA-256 demonstrates the principle of never storing reversible plaintext credentials, but it is a fast, general-purpose hash rather than a password-hashing function. Without a per-user salt, identical passwords produce identical stored hashes, and the digests remain vulnerable to a precomputed dictionary attack if users.json is ever disclosed. Priority-2 remediation: migrate to bcrypt or Argon2 (Section 15.2).

6.3 Duplicate Login Prevention

The server maintains an in-memory set, logged_in_users, of usernames with an active session. A login attempt for a username already present in that set is rejected with DUPLICATE_LOGIN before any session state is created, and the event is logged. This prevents the same account from being used concurrently from two client processes, which would otherwise complicate message routing (which of two sockets for the same username should receive a private message?) and session-timeout bookkeeping.

Figure 6.3 — Duplicate Login Rejection

Client-side dialog shown when a second connection attempts to authenticate with a username that already has an active session.

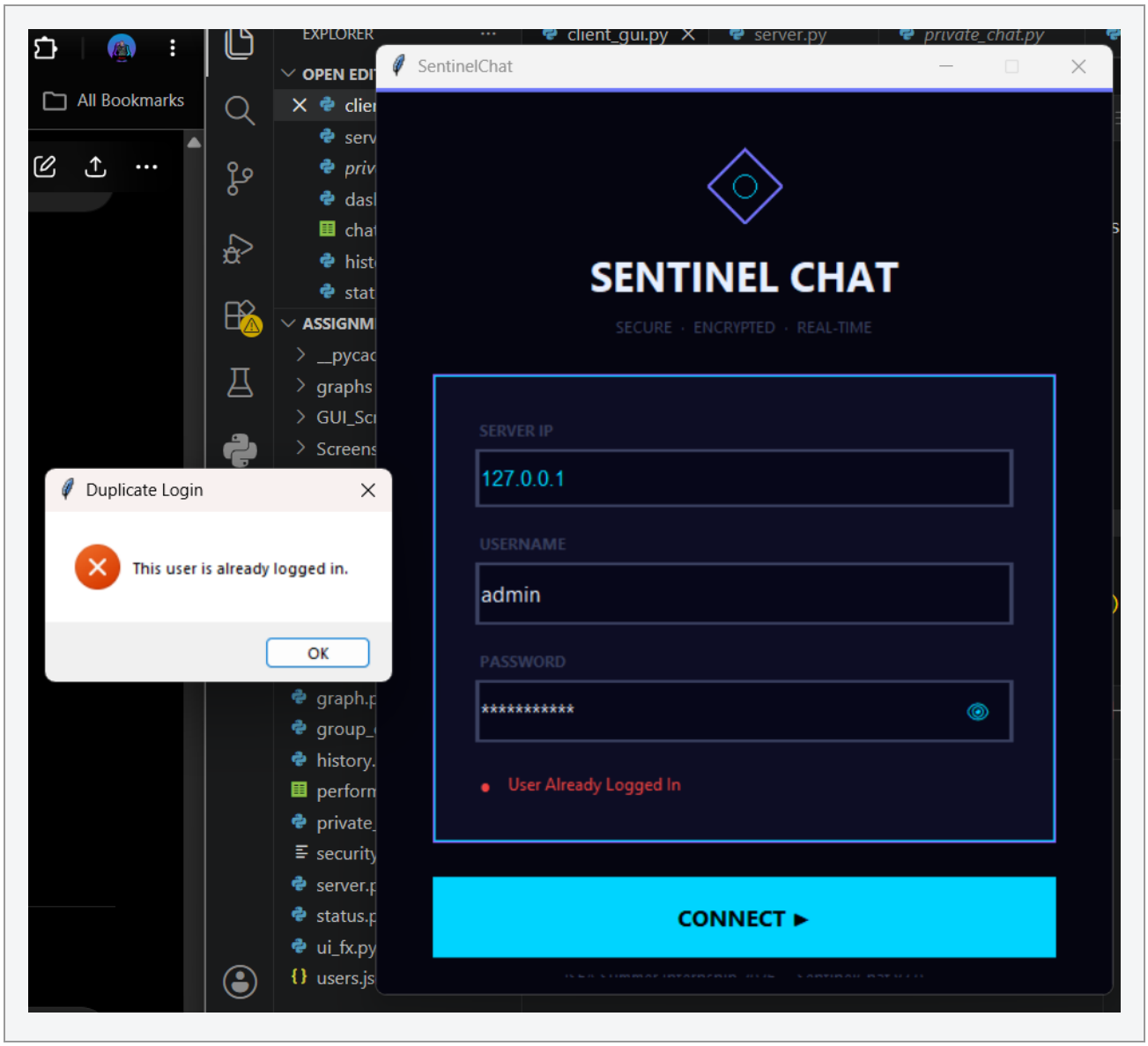

Figure 6.3 — DUPLICATE_LOGIN rejected before any new session state is created.

6.4 Failed-Login Protection and Account Lockout

The server tracks consecutive failed attempts per username in a failed_attempts dictionary. On the fifth consecutive failure (MAX_LOGIN_ATTEMPTS = 5, from config.json), the account is placed into blocked_users with an expiry timestamp computed as now + LOCK_TIME (300 seconds by default) and the client is sent ACCOUNT_LOCKED. Subsequent login attempts for a locked account are rejected immediately, without re-attempting authentication, until the lockout window elapses, at which point the failure counter is reset.

Client-side countdown discrepancy: the login window displays a 30-second visual lockout countdown independent of the server's actual 300-second LOCK_TIME value. This is a genuine inconsistency between client presentation and server enforcement — the button re-enables and invites a retry five minutes before the server will actually accept it — and is listed as a defect to correct in Section 14, not treated as intended behavior.

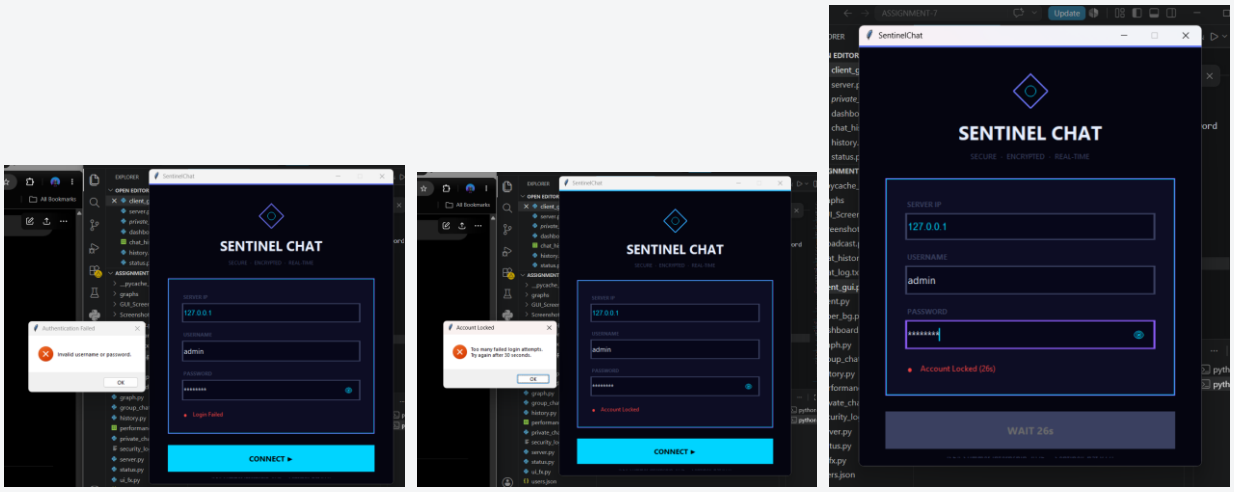

Figure 6.2 — Account Lockout and Countdown

(a) A rejected login attempt, (b) the ACCOUNT_LOCKED dialog, and (c) the lockout wait-count indicator.

Figure 6.2 — Failed-login handling and lockout countdown.

6.5 Session Timeout

Each client handler thread calls client.settimeout(1) and loops on recv(), catching socket.timeout to periodically check elapsed idle time against SESSION_TIMEOUT (180 seconds by default) without blocking indefinitely on a silent connection. When the threshold is exceeded, the server logs SESSION TIMEOUT, sends the literal string SESSION_TIMEOUT to the client, and closes the connection. The GUI recognizes this exact payload in Dashboard._receive_loop() and transitions the user back to the login window with an explanatory dialog, rather than presenting a raw connection error.

Figure 6.4 — Session Timeout

Client-side dialog and idle-timeout indicator shown when the server closes a connection after SESSION_TIMEOUT seconds of inactivity.

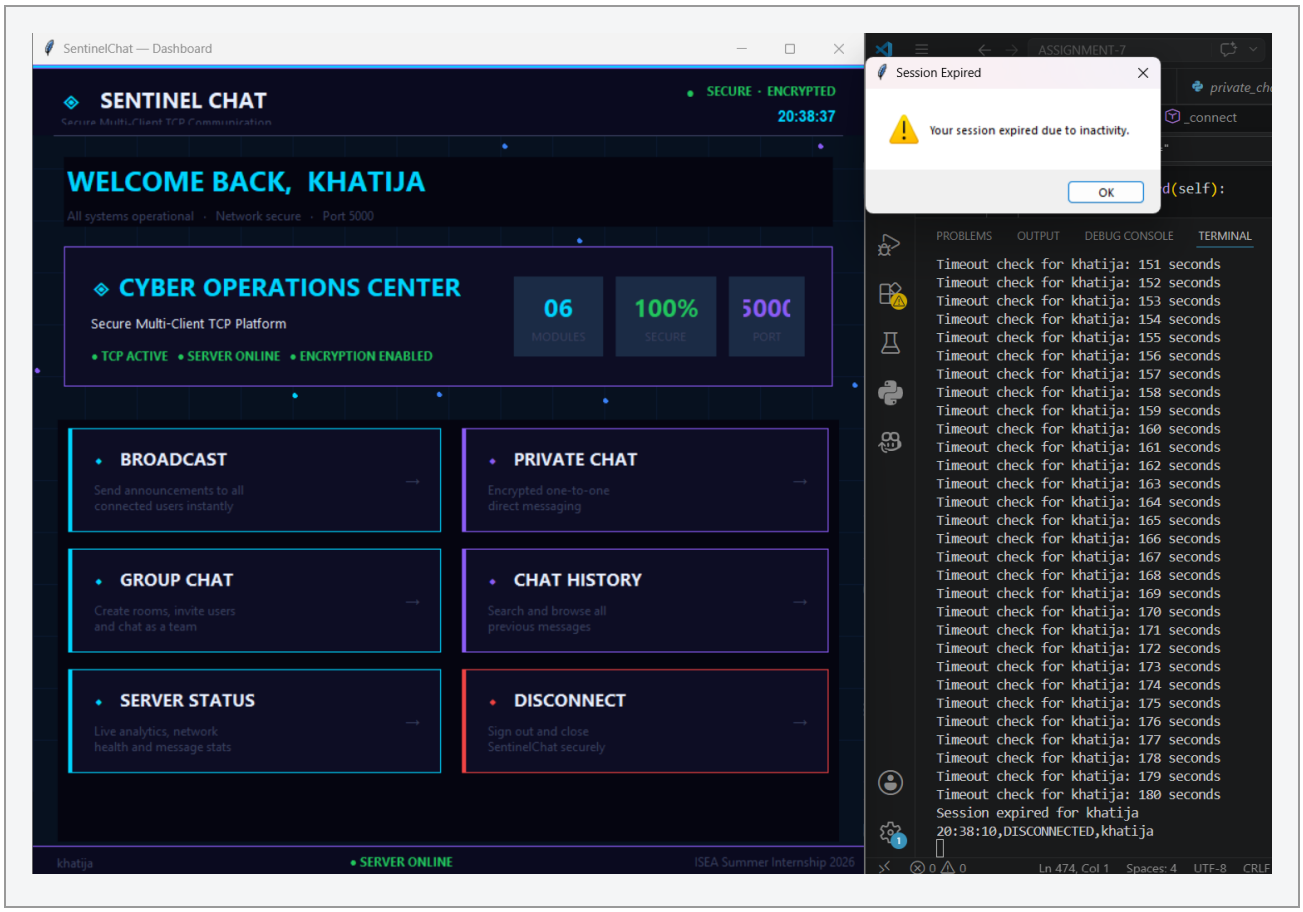

Figure 6.4 — Idle session closed by the server; the client returns to the login window.

6.6 Security Event Logging

Every authentication-relevant event — LOGIN_SUCCESS, LOGIN FAILED, DUPLICATE LOGIN, ACCOUNT LOCKED, SESSION TIMEOUT, and LOGOUT — is appended to security_log.txt with a timestamp, via a single security_log(event, username) helper. Passwords are never written to this file at any point. The accumulated log (reviewed as part of this report) confirms the mechanism is active and has recorded genuine multi-user test sessions across several dates in July 2026.

Data-quality observation: the accumulated security_log.txt contains a duplicated line for every successful login (LOGIN_SUCCESS immediately followed by an identically timestamped LOGIN_ SUCCESS with an extra space), and one username appears with inconsistent capitalization (Fathima vs. fathima) in a single entry. Neither issue affects the authentication decision itself, since comparisons elsewhere in the server are case-sensitive and consistent within a session, but both are logging-quality defects that should be corrected — see Section 14.

6.7 Input Validation

The server rejects malformed login packets (anything not matching the three-part LOGIN|user|pass format) and empty username/password fields are rejected client-side before a connection attempt is made. Command parsing on already-authenticated connections is prefix-based and tolerant: an unrecognized command falls through to being treated as an unscoped chat message rather than raising an exception, which favors availability over strict protocol enforcement. Message length is not capped at the server; the 500-character limit enforced in the Broadcast window (Section 7.4.1) is a client-side courtesy limit only, not a server-side guarantee.

6.8 Summary of Security Posture

The mechanisms above provide meaningful protection against casual account sharing, brute-force password guessing, and unattended-session exposure within a trusted network. They do not provide confidentiality of message content or credentials in transit: all data — including the LOGIN|user|password packet — is sent as plaintext over TCP and is fully readable in a Wireshark capture (Section 12.2, Figure 12.2). This is the single most significant security limitation in the current system and is discussed with a concrete remediation path in Sections 14 and 15.

7. Client Application Architecture

7.1 Design Philosophy

The client is a Tkinter desktop application styled with a consistent dark, high-contrast "cyber operations" theme (near-black backgrounds, cyan #00D4FF primary accent, purple #8B5CF6 secondary accent) applied uniformly across every window. The visual system exists to give each message type and application state (connected, locked, error, success) an unambiguous color identity, and is implemented entirely with native Tkinter primitives — Canvas, Frame, and Text widget tag configuration — with no external GUI toolkit or graphics library dependency.

7.2 Login Window

client_gui.py presents server IP, username, and password fields, a show/hide password toggle, and a connection-status indicator. On submission it performs up to three connection attempts (with a short delay between attempts) before reporting failure, which tolerates transient connection issues (e.g., a server still finishing its bind/listen call) without requiring the user to manually retry. Successful authentication tears down the login window and constructs the Dashboard, passing it the authenticated username, the live socket, and the server IP.

7.3 Dashboard — Central Receiver and Navigation Hub

dashboard.py is the architectural center of the client. It is the only component that owns the socket's background receive thread; every other window is a pass-through consumer of messages the dashboard hands it. This is described fully in Section 8.2. In addition to message dispatch, the dashboard renders the six-card feature grid (Broadcast, Private Chat, Group Chat, Chat History, Server Status, Disconnect), a live clock, an animated heartbeat indicator, and hosts the reconnect workflow used when the socket fails unexpectedly.

Figure 7.1 — SentinelChat Dashboard (Cyber Operations Center)

Dashboard showing the six feature cards, hero panel, and live clock/heartbeat indicator.

Figure 7.1 — Main dashboard and navigation hub.

7.4 Feature Modules

7.4.1 Broadcast (broadcast.py)

A dedicated composer window for /all messages, with a live character counter (visual warning past 500 characters) and a transient confirmation banner on send. The 500-character limit is enforced only in this window; it is not a protocol-level constraint (Section 6.7).

Figure 7.4 — Broadcast Composer

The Broadcast window: recipient scope (all connected users), message field, and live character counter.

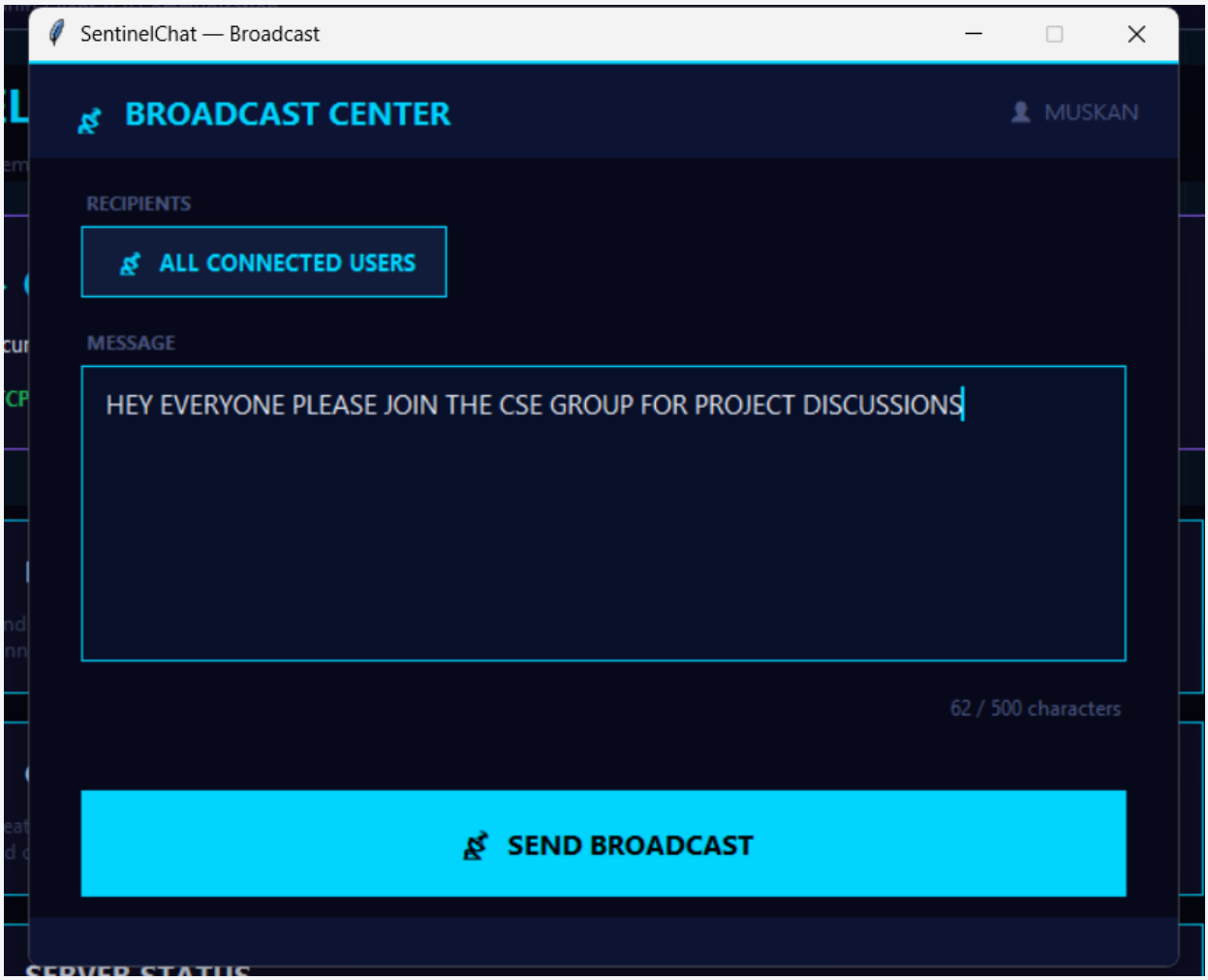

Figure 7.4 — Broadcast composer window used for /all messages.

7.4.2 Private Chat (private_chat.py)

Presents an online-user list (populated by requesting /list on open) alongside a WhatsApp-style bubble chat view rendered through the shared ui_fx helpers. Selecting a user establishes the addressing context for subsequent /msg sends; incoming [PRIVATE] messages are routed to this window by the dashboard dispatcher regardless of which window currently has focus.

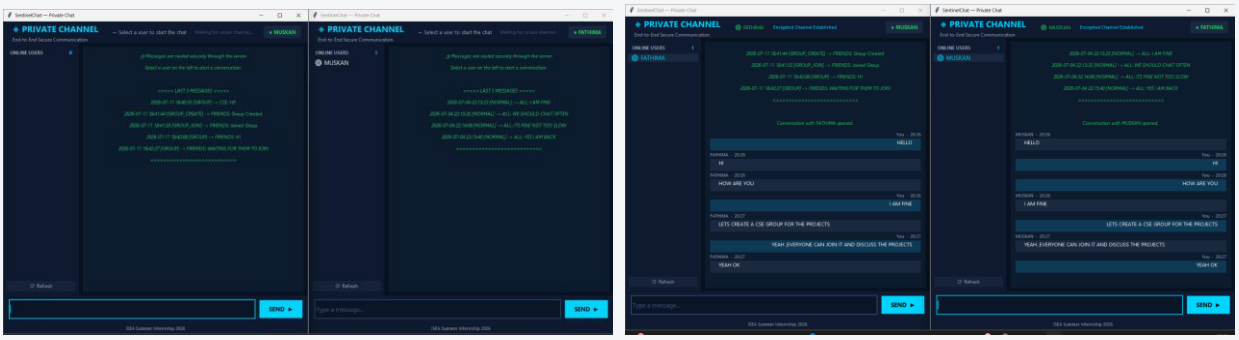

Figure 7.2 — Private Chat Interface

Private chat windows from two simultaneous client sessions, showing the online-user list and bubble-style message thread.

Figure 7.2 — One-to-one encrypted-looking (transport-plaintext) chat interface.

7.4.3 Group Chat (group_chat.py)

Supports creating a new named group, joining an existing group, and sending messages scoped to a group. Group membership as seen by this window is client-local (a list populated from the user's own create/join actions); it is not queried back from the server, so a user has no way to discover which groups exist server-side without being told the name out of band. This is a usability limitation, not a security one — group messages are still routed correctly by the server regardless of what the client's local list shows.

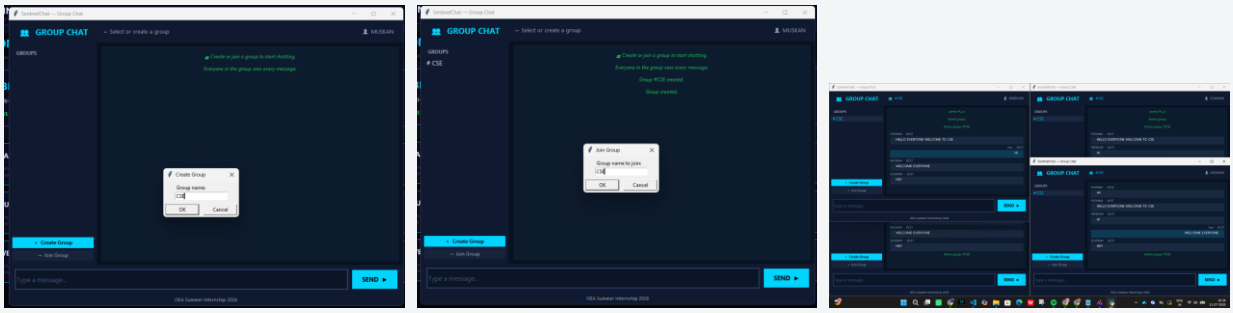

Figure 7.3 — Group Chat Interface

Group creation, joining, and a multi-member conversation thread.

Figure 7.3 — Group creation, joining, and shared messaging.

7.4.4 Chat History (history.py)

Reads chat_history.csv directly from local disk (not via the socket) and provides free-text search plus a message-type filter (All / Private / Broadcast / Group). Because it reads the file directly, this window shows the same history file the server has been continuously appending to for every session on that machine, not just the current session's messages.

Figure 7.5 — Chat History Viewer

The Chat History window with free-text search and the All / Private / Broadcast / Group type filter applied.

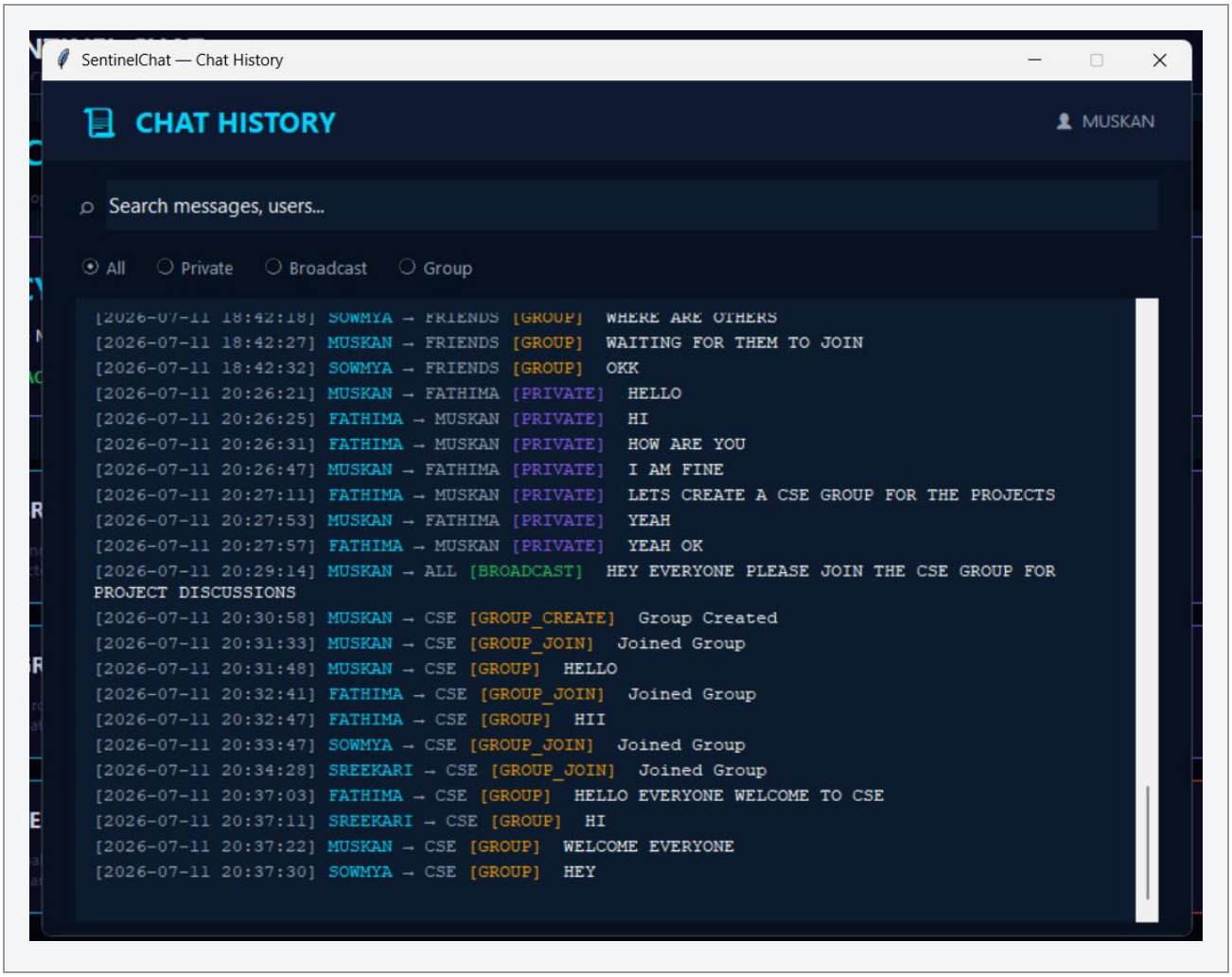

Figure 7.5 — Chat History viewer, reading chat_history.csv directly from local disk.

7.4.5 Server Status (status.py)

Displays four summary counters (connected users, total messages, broadcasts, private messages) and two horizontal utilization bars labeled CPU and MEM, refreshed every three seconds. The message counters are genuine, computed by re-scanning chat_history.csv. The connected-users counter and the CPU/MEM bars are not: they are generated with random.randint() and do not read any real process or OS telemetry. This window is therefore a UI mockup of a monitoring dashboard rather than a functioning one for those three values, and is documented as such rather than represented as a completed monitoring feature. See Section 14 for the recommended remediation (e.g., psutil-based sampling and a server-reported connected-client count).

Figure 7.6 — Server Status Window

The Server Status window: four summary counters and the CPU / MEM utilization bars discussed in Section 7.4.5.

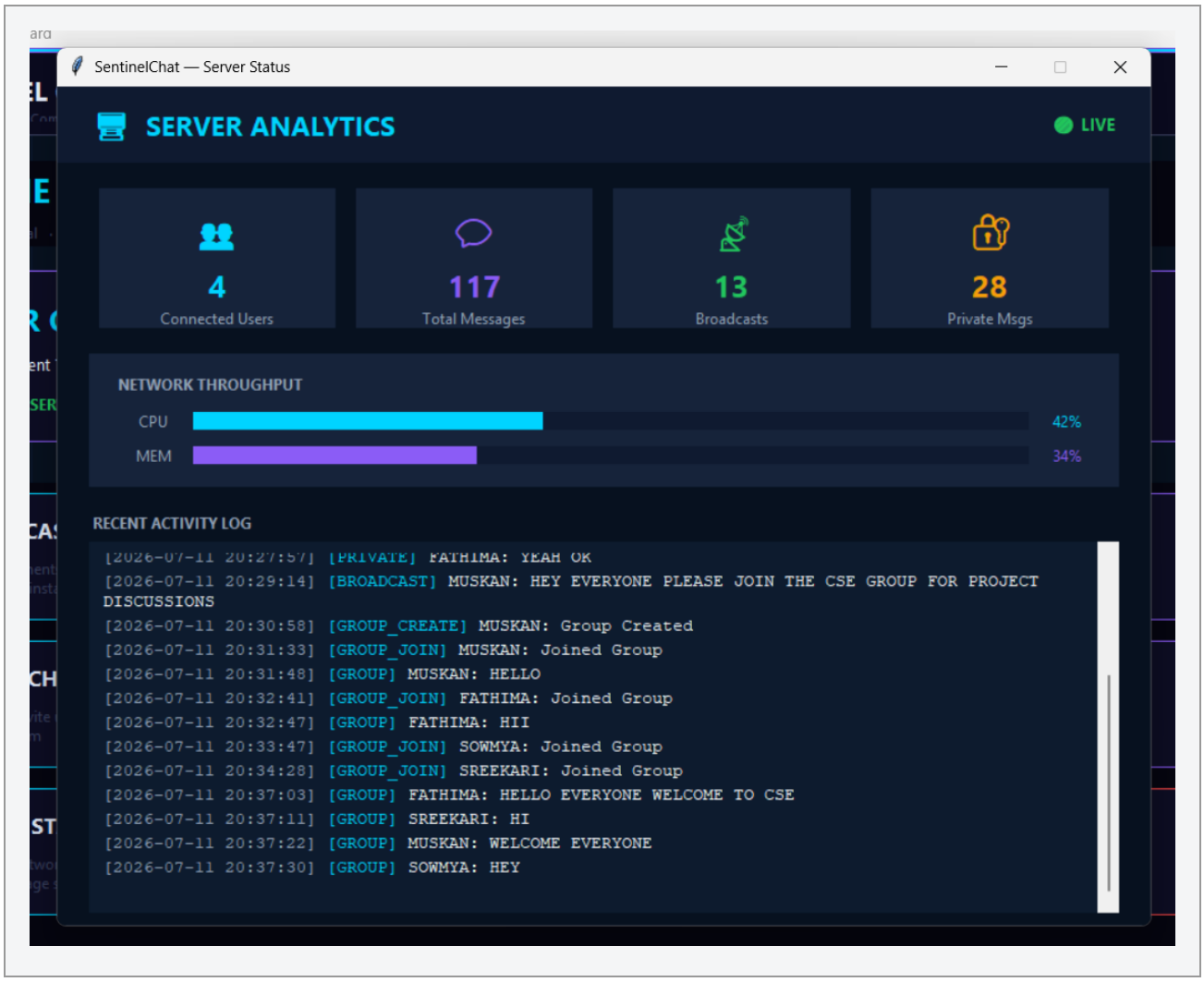

Figure 7.6 — Server Status dashboard. Connected-users and CPU/MEM values are simulated, not measured; message counters are genuine.

7.5 Visual Effects Subsystem

cyber_bg.py implements a lightweight animated particle background (40 drifting dots redrawn via Canvas.after()) used on the dashboard, and ui_fx.py provides three reusable primitives shared by the private-chat and group-chat windows: GlowPulse (a sine-wave color interpolation used for a "breathing" input-field border), add_bubble_tags/add_bubble (WhatsApp-style right/left-aligned message rendering using native Text widget paragraph tags, avoiding a custom Canvas-based chat renderer), and add_system (centered italic system notices). These modules contain no networking calls, which keeps them independently testable and reusable across windows without any risk to message-delivery correctness.

8. Threading and Concurrency Model

8.1 Server-Side Threading

The server's accept loop (receive() in server.py) runs on the main thread. For each successfully authenticated connection, it spawns one daemon thread executing handle_client(client), which owns that connection's request loop for its entire lifetime. Shared mutable state — the clients list in particular — is guarded by a threading.Lock (clients_lock) during broadcast iteration, preventing a race between a client disconnecting mid-broadcast and the list being iterated. Other shared dictionaries (usernames, groups, client_info) are mutated from multiple threads without an explicit lock; in CPython, the Global Interpreter Lock makes individual dict operations atomic enough that this has not produced observed corruption in testing, but it is not a formally correct concurrency guarantee and is noted as a hardening item in Section 14.

8.2 Client-Side Dispatcher — the Single-Receiver Optimization

This is the most important architectural decision in the client and is described here in full because it directly explains why several other modules look the way they do. In an earlier iteration of the client design, each feature window that needed to receive messages (private chat, group chat) implemented its own _receive_loop() method that called client.recv() in a dedicated thread. Because all feature windows share the same underlying TCP socket, having more than one thread call recv() on it concurrently is unsafe: only one of the competing threads would ever receive a given inbound message, non-deterministically, and a message intended for the group-chat window could be silently consumed by the private-chat window's thread instead.

The final architecture resolves this by giving exactly one component — dashboard.py — ownership of the socket's receive thread. Dashboard._receive_loop() is the only method in the entire client that calls client.recv(). Every message it receives is handed to Dashboard._dispatch_message(), which inspects the message prefix ([PRIVATE], [GROUP ..., [BROADCAST], or Online Users:) and forwards it to the correct open window via a direct method call (private_chat.receive_message(msg), group_chat.receive_message(msg)), or discards it if the relevant window is not currently open. Because this call happens inside root.after(0, ...), it always executes on the Tkinter main thread, so the destination window's widget updates are thread-safe by construction.

> **ENGINEERING NOTE — Single-Receiver Design**
> Only dashboard.py ever calls client.recv(); every other window receives messages exclusively through Dashboard._dispatch_message(). This single-ownership rule is what eliminates the socket race condition that the earlier per-window receive-loop design would have reintroduced.

The vestigial _receive_loop() methods still present in private_chat.py and group_chat.py (commented out at their call sites) are remnants of the earlier per-window design. They are not started and have no effect on current behavior, but their presence in the codebase is a maintainability defect: a future contributor could re-enable one by mistake and reintroduce the exact race condition this architecture was built to eliminate. Removal is recommended in Section 14.

8.3 Thread-Safety of UI Updates

Tkinter's mainloop() is not thread-safe with respect to arbitrary background threads calling widget methods. Every place in the client where a background thread (the dashboard's receive thread, or the login window's connection-retry loop) needs to update a widget, it does so exclusively through root.after(0, callback, ...), which schedules the

callback to run on the main thread's event loop rather than executing it immediately from the calling thread. This single convention is applied consistently across client_gui.py and dashboard.py and is the reason the GUI does not freeze or corrupt its display state while waiting on network I/O.

9. Data Persistence and Configuration

SentinelChat uses flat files exclusively for persistence — JSON for structured configuration and credentials, CSV for tabular history and performance data, and plain text for the append-only security log. No relational or embedded database is used. This is an appropriate choice for the system's current scale and purpose (a single-server demonstration application) and keeps every stored artifact human-readable and directly inspectable, which materially simplified the testing and verification work described in Sections 11 through 13.

9.1 config.json — Runtime Configuration

Key	Purpose	Observed Value
SERVER_HOST	Bind address for the server socket	127.0.0.1
SERVER_PORT	TCP port for the server socket	5000
BUFFER_SIZE	recv() buffer size in bytes	4096
SESSION_TIMEOUT	Idle seconds before automatic logout	180
MAX_LOGIN_ATTEMPTS	Consecutive failures before lockout	5
LOCK_TIME	Lockout duration in seconds	300
MAX_CLIENTS	Listen backlog / intended concurrent-client ceiling	20

All seven values are read once at process start via a single json.load() call; there is currently no support for reloading configuration without restarting the server, which is an acceptable trade-off for a system of this scale.

9.2 users.json — Credential Store

A flat mapping of username to a single "password" field holding a SHA-256 hex digest (Section 6.2). The file currently defines five demonstration accounts (admin, khatija, fathima, sowmya, sreekari). As noted in Section 6.2, all five currently share an identical hash value, which is consistent with a shared demo password used across test accounts rather than distinct credentials — this should be stated explicitly in any external-facing description of the system's security testing so that reviewers do not assume five independently verified credentials were exercised.

9.3 chat_history.csv — Message Log

A single, continuously-appended CSV with five columns: timestamp, sender, receiver, message_type, message. Every message the server routes — NORMAL, PRIVATE, BROADCAST, GROUP, GROUP_CREATE, and GROUP_JOIN — is logged here exactly once by log_chat(), server-side, regardless of which client sent it. This guarantees a single, consistent history file per server instance rather than requiring reconciliation of per-client logs.

Data-quality observation: the accumulated file contains at least one malformed row with fewer than five comma-separated fields (a truncated GROUP_JOIN entry), which a naive csv.DictReader consumer would either mis-map or skip depending on the reader's error tolerance. history.py's reader filters blank lines but does not explicitly validate field count, so a row like this is a latent robustness gap. This is listed as a defect to fix in Section 14 rather than treated as expected behavior.

9.4 security_log.txt — Security Audit Trail

Append-only, human-readable, one event per line in the form [YYYY-MM-DD HH:MM:SS] EVENT : username. Reviewed log data spans multiple real testing sessions across mid-to-late July 2026 and shows realistic sequences (repeated failed logins culminating in a lockout, followed by a successful login after the lockout window elapsed), which corroborates that the lockout and timeout mechanisms described in Section 6 were exercised under real conditions rather than only unit-tested in isolation. The duplicated-line and case-inconsistency observations from Section 6.6 apply to this file specifically.

9.5 performance_results.csv — Performance Samples

Five columns — clients, broadcast_messages, private_messages, avg_delay_ms, throughput_msgs_per_sec — with one row per tested client-count. The current file contains three rows (2, 3, and 4 concurrent clients). This dataset is discussed critically in Section 11; it is listed here purely as a persistence artifact.

10. Reliability and Resource Management

This section documents the reliability engineering introduced in the final development stage (Section 3.5): mechanisms that keep the server stable and its resource usage bounded as clients connect, disconnect, and occasionally fail ungracefully, over the lifetime of a long-running server process.

10.1 Automatic Client Cleanup

cleanup_client(client) is the single point through which a client is removed from server state, whether the removal is triggered by a deliberate disconnect, a session timeout, or an unhandled exception in that client's handler thread. It closes the socket (tolerating an already-closed socket via a caught OSError), and removes the client from clients, usernames, logged_in_users, client_info, and every entry in groups that referenced it. Centralizing this logic in one function — rather than duplicating partial cleanup at each of the several call sites that can end a session — was a deliberate choice to prevent the class of bug where a client is removed from one tracking structure but left as a stale reference in another (which would otherwise manifest later as a send() to a dead socket during broadcast).

10.2 Socket Reuse — SO_REUSEADDR

The server socket is created with socket.SO_REUSEADDR enabled before binding. Without this option, restarting the server shortly after a previous instance exits can fail with "address already in use" while the OS holds the port in a TIME_WAIT state. Enabling it allows the server to be restarted immediately during iterative development and testing — a small but practically significant reliability improvement for the development workflow, distinct from any client-facing behavior.

10.3 Client-Side Reconnection

If the dashboard's receive thread encounters a socket exception that isn't a deliberate user-initiated disconnect, Dashboard._reconnect() is invoked: it opens a modal progress dialog and probes the server up to three times with a short timeout, informing the user of the outcome and routing them back to the login window whether or not the probe succeeds. This gives the user actionable feedback ("Connection Restored — please log in" vs. "Reconnect Failed — please log in again") rather than leaving a frozen or silently dead window, though it should be noted this is a manual reconnect-and-relogin flow, not a transparent session-resumption mechanism — the user's socket and session state are not preserved across a reconnect.

Figure 10.1 — Disconnect Action

The dashboard's Disconnect control, the deliberate exit path into the same teardown that an unexpected socket failure also triggers.

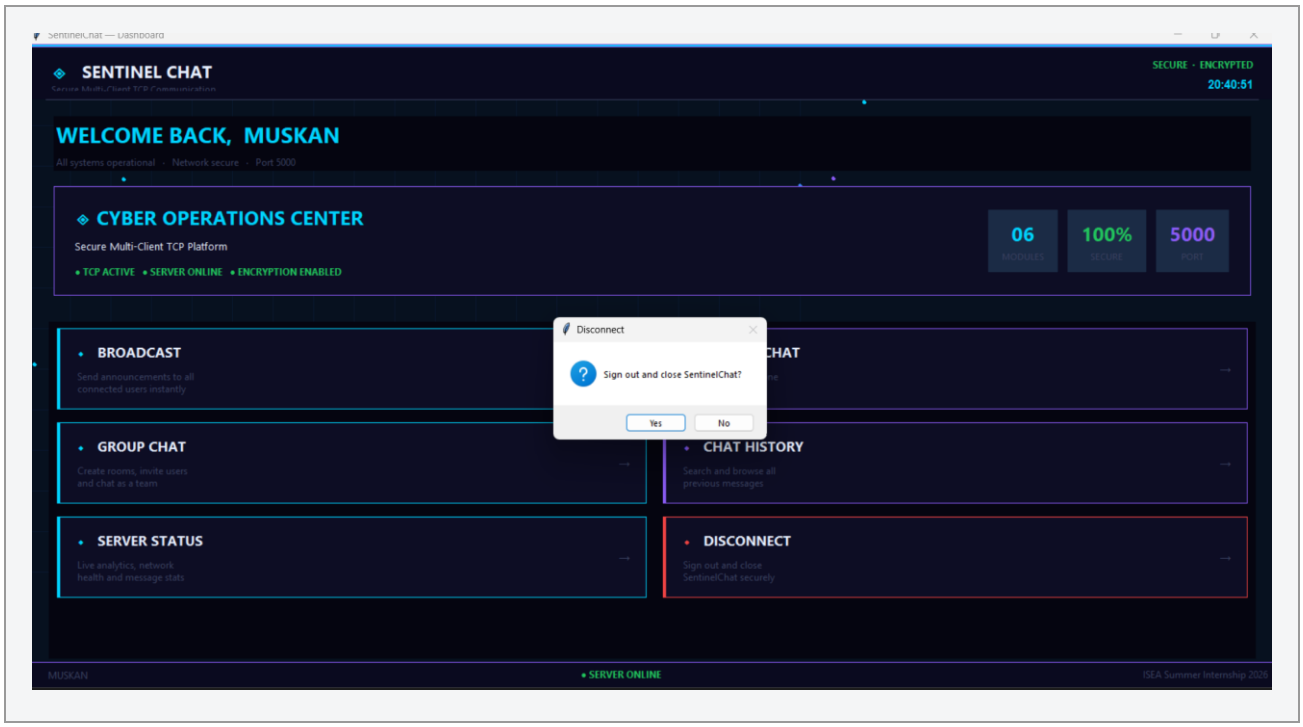

Figure 10.1 — Disconnect action; an unexpected socket failure routes through the same reconnect-and-relogin flow described in Section 10.3.

10.4 Exception Handling Pattern

The server applies a consistent pattern around every send() call in its routing functions: attempt the send, and on ConnectionResetError, BrokenPipeError, or OSError, treat it as evidence that the peer is gone and call cleanup_client() rather than propagating the exception. The per-client handler loop applies a broader bare except around its main recv() call to guarantee that any unexpected condition on one client's connection results in that client being cleaned up and its thread exiting, without taking down the accept loop or any other client's thread. This favors availability of the overall service over surfacing detailed error diagnostics to the operator — an appropriate trade-off for this application, though production hardening would typically add structured error logging alongside the current behavior (see Section 15).

10.5 Graceful Shutdown

On KeyboardInterrupt at the server console, the server prints summary statistics (private message count, broadcast message count, groups created, connected clients) and then closes every tracked client socket before closing the listening socket. This provides an operator-visible summary at shutdown and avoids leaving client sockets in an undefined state when the server process exits deliberately.

11. Performance Evaluation

A performance-measurement pipeline was built to characterize how average message delay and message throughput change as the number of concurrently connected clients increases. This section presents the pipeline, the data it has produced to date, and an honest assessment of what conclusions that data does and does not support.

11.1 Methodology

Client instances (2, 3, and 4 simultaneous connections) exchanged broadcast and private messages against the running server; observed broadcast_messages and private_messages counts, average delay, and throughput were recorded per run into performance_results.csv. graph.py then reads this file with pandas and renders two line charts (clients vs. average delivery time, clients vs. throughput) with matplotlib, plus a bar chart of message-type distribution.

11.2 Results

Clients	Broadcast Msgs	Pr M
2	20	5
3	30	8
4	40	10

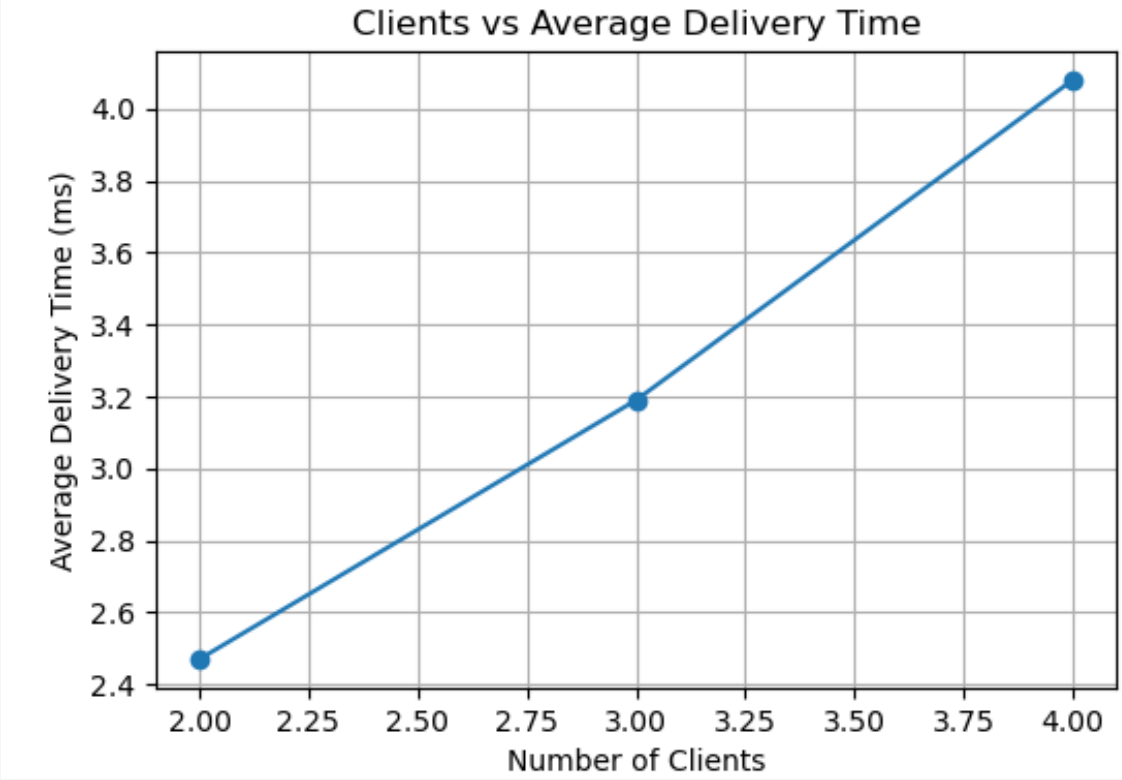

Figure 11.1 — Clients vs. Average Delivery Time

Generated by graph.py from performance_results.csv.

Figure 11.1 — Average delay increases modestly and roughly linearly with client count over the tested range.

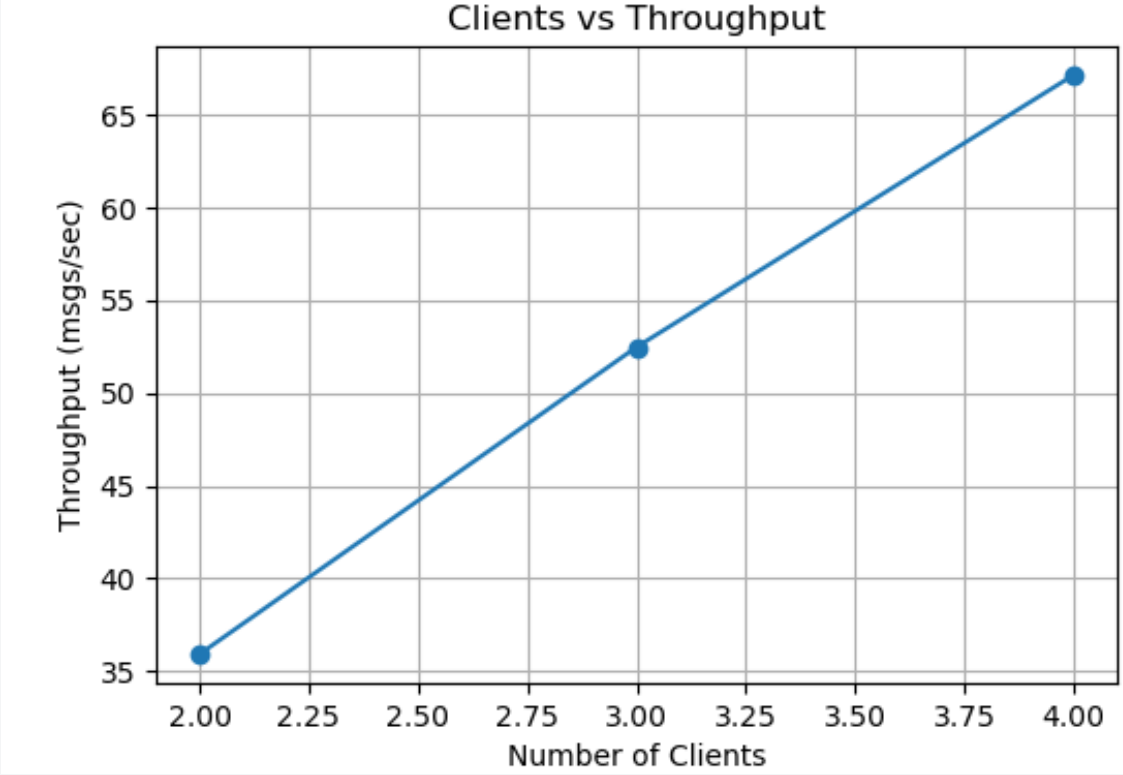

Figure 11.2 — Clients vs. Throughput

Generated by graph.py from performance_results.csv.

Figure 11.2 — Throughput increases with client count over the tested range, consistent with more total messages being generated per unit time rather than a per-message speed improvement.

11.3 Interpretation

Within the tested range of 2–4 clients, average delay increases from 2.47 ms to 4.08 ms and throughput increases from roughly 36 to 67 messages per second. The throughput increase is expected and largely mechanical: more connected clients generate more total message traffic, so aggregate messages-per-second rises even if the server's per-message handling cost is constant or slightly increasing. The delay increase is consistent with additional contention for the shared clients_lock during broadcast fan-out as client count grows.

11.4 Message-Type Distribution Figure — Data-Quality Note

graph.py's message-type distribution bar chart is currently generated from a fixed literal array ([5, 1, 3, 1] for Normal, Private, Broadcast, and Group) rather than being computed from chat_history.csv or performance_results.csv. This chart should be treated as illustrative placeholder output only, and must not be cited as a measured message-type breakdown until the script is corrected to compute real counts from the accumulated history file. This is listed as a required fix in Section 14.

Figure 11.3 — Message-Type Distribution (Illustrative Only)

Output of graph.py's message-type distribution chart, included here for completeness. As Section 11.4 explains, this chart is currently generated from a fixed literal array rather than computed from chat_history.csv, and must not be cited as a measured breakdown.

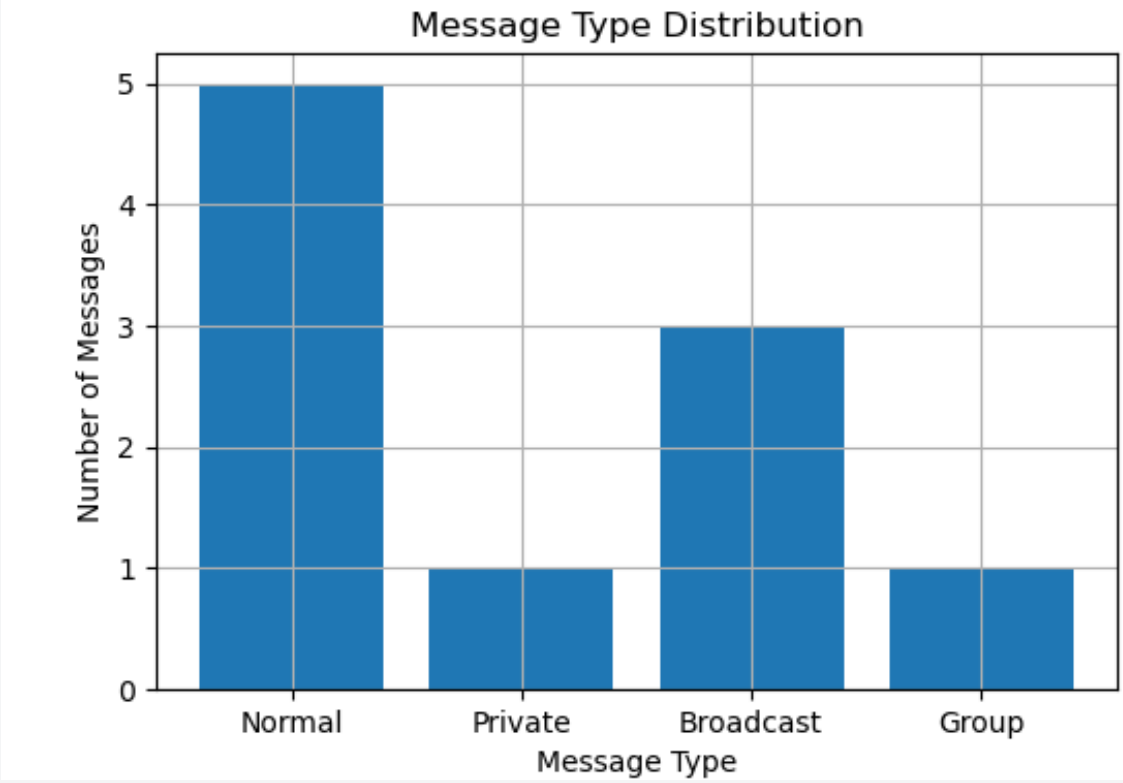

Figure 11.3 — Illustrative pipeline output only; not yet computed from real history data.

11.5 Honest Assessment of Evidentiary Strength

Three data points, collected in a single test run without repetition, is sufficient to demonstrate that the measurement pipeline (instrumentation → CSV → chart generation) works end to end, but it is not a statistically defensible performance benchmark. There is no recorded test methodology for how messages were generated (message size, send rate, test duration), no variance or repeated-trial data, and — critically — no "before optimization" dataset from prior to the Assignment 8 configuration/cleanup changes to compare against. Consequently, this report does not claim that the Assignment 8 optimizations measurably improved delay or throughput; it claims only that the measurement pipeline exists and functions, and that the three collected data points are internally consistent with expected TCP broadcast behavior at small scale. A rigorous before/after benchmark is listed as recommended follow-up work in Section 15.

Separately, the CPU and memory utilization figures presented in the live Server Status window (Section 7.4.5) are not derived from this pipeline or from any real system sampling — they are randomly generated for visual demonstration purposes and must not be cited as performance evidence in any context.

12. Network Verification — Wireshark

12.1 Methodology

TCP traffic was captured on the loopback interface using Wireshark with the display filter tcp.port == 5000. Because the underlying socket protocol has not changed since it was first verified in Assignment 4, and the GUI client (Assignment 6 onward) sends exactly the same command strings the original terminal client sent, the packet-level behavior verified at each stage remains valid for the completed system — this report does not re-derive separate Wireshark evidence per historical assignment, only a single consolidated verification record.

12.2 Verified Behaviors

Capture	What It Confirms
TCP three-way handshake	SYN -> SYN-ACK -> ACK exchange when a client connects, confirming standard TCP connection establishment on port 5000.
Login / authentication traffic	A PSH/ACK segment carrying the LOGIN|user|password payload in plaintext, and the server's LOGIN_SUCCESS / LOGIN_FAILED response — this capture is also the concrete evidence for the plaintext-transport limitation discussed in Sections 6.8 and 14.
Failed login attempt	PSH/ACK segment carrying a rejected login, correlated with the corresponding LOGIN FAILED entry in security_log.txt.
Broadcast message	PSH/ACK segment carrying an /all payload, visible on

	every connected client's stream.
Private message	PSH/ACK segment carrying a /msg payload, visible only on the sender's and the named recipient's TCP streams — confirming server-side routing restricts delivery as designed.
Group message	PSH/ACK segment carrying a /group payload, visible only on the streams of clients that had issued /group join for that group.
Connection termination	FIN/ACK teardown sequence when a client disconnects via the Disconnect action or session timeout.

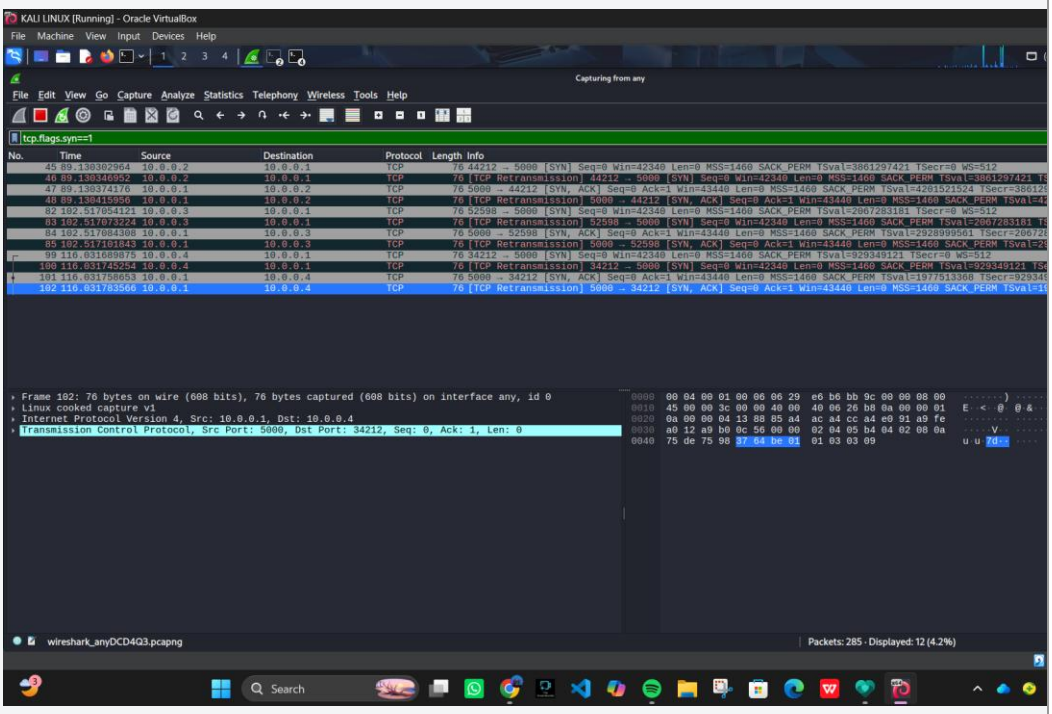

Figure 12.1 — TCP Three-Way Handshake on Port 5000

Wireshark capture showing the SYN / SYN-ACK / ACK exchange.

Figure 12.1 — Connection establishment.

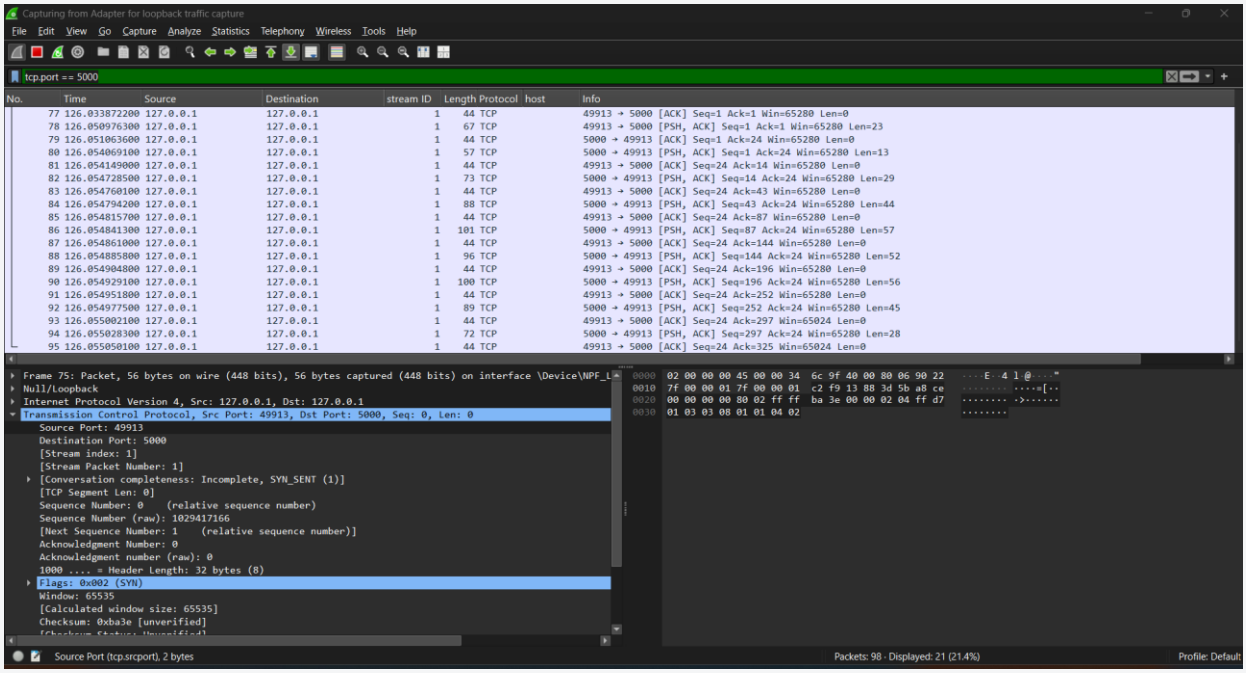

Figure 12.2 — Login Traffic (Plaintext Credential Payload)

Wireshark capture of the LOGIN|user|password PSH/ACK segment, with the plaintext payload visible in the packet bytes pane — documentary evidence of the transport-encryption limitation.

Figure 12.2 — Authentication payload as observed on the wire.

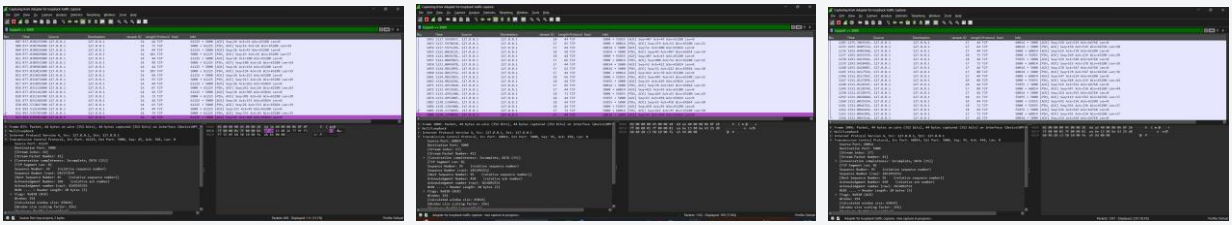

Figure 12.3 — Broadcast, Private, and Group Message Capture

Wireshark captures corresponding to /all, /msg, and /group traffic.

Figure 12.3 — Server-side routing confirmed at the packet level.

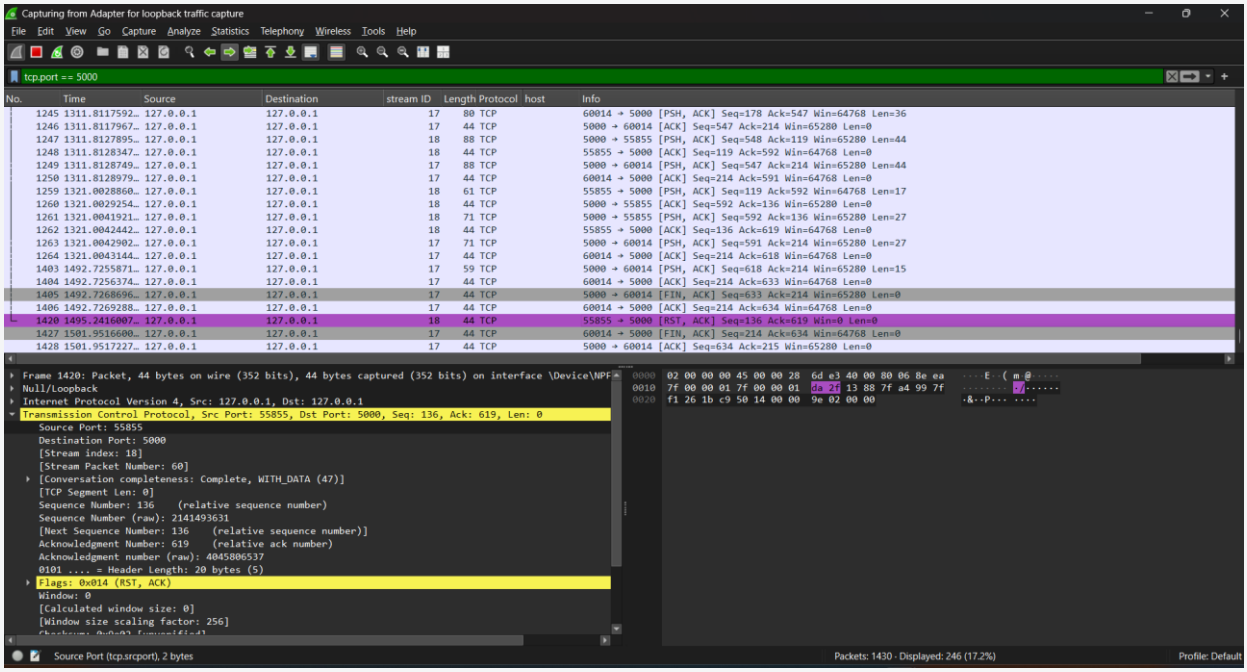

Figure 12.4 — Connection Termination

Wireshark capture of the FIN/ACK teardown on client disconnect.

Figure 12.4 — Graceful connection teardown.

12.3 Conclusion of Network Verification

Across every capture reviewed, application traffic is consistently carried over TCP port 5000 with no unexpected side-channels, and the packet-level evidence directly corroborates the routing behavior described in Section 5 and the security event log described in Section 6.6. The capture evidence also directly demonstrates, rather than merely stating, the plaintext-transport limitation: the login packet's payload bytes are human-readable in the packet detail pane, which is the clearest available justification for prioritizing TLS as the top item in the future-enhancements roadmap (Section 15).

13. Software Testing

Testing was performed manually, using multiple simultaneous client instances against a running server, rather than through an automated test suite. This section consolidates the functional and security test evidence gathered across development into a single matrix and states plainly where automated coverage does not yet exist.

Figure 13.1 — Multi-Client Functional Test

A multi-client test session of the kind used to build the functional test matrix in Section 13.1.

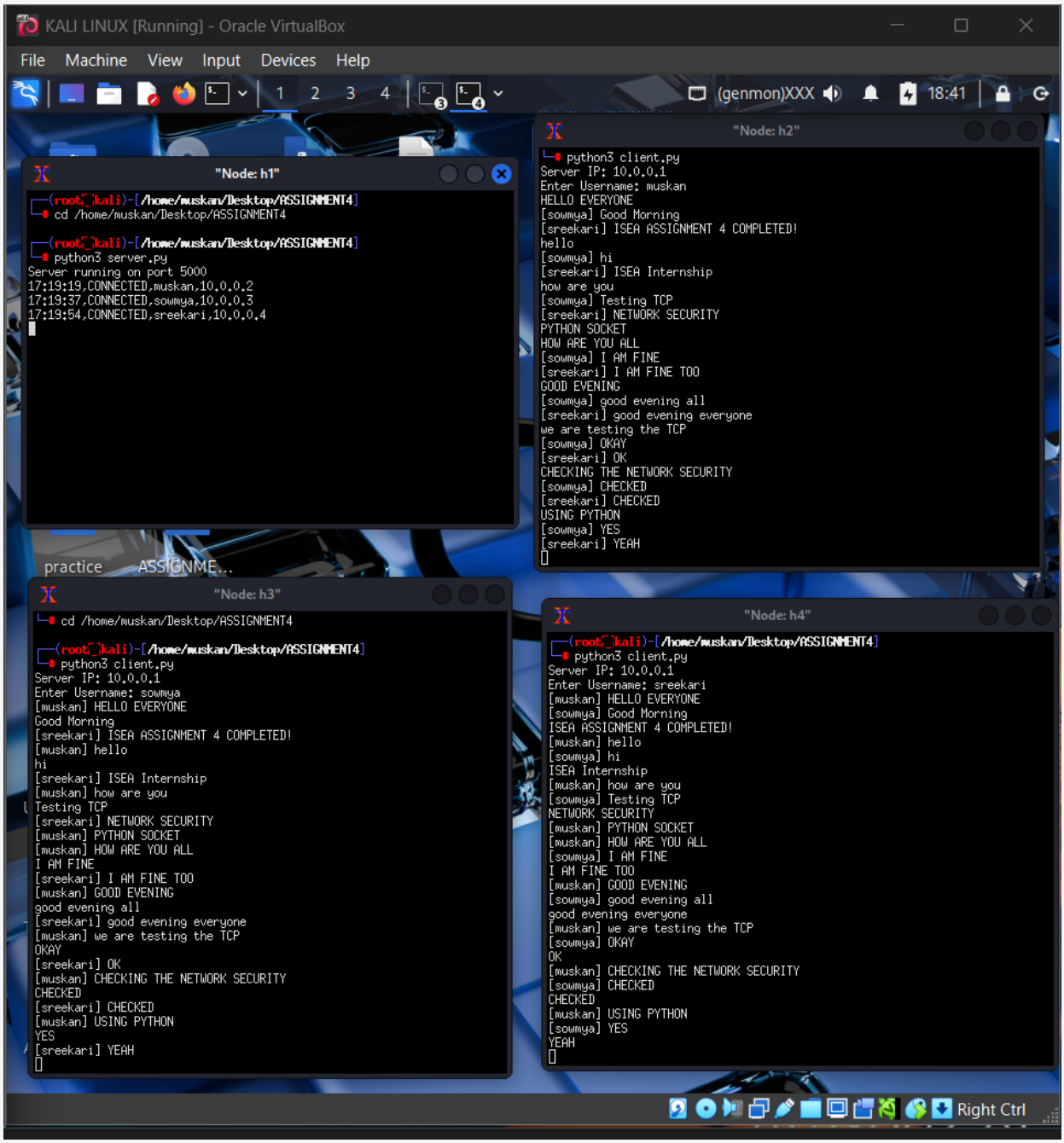

Figure 13.1 — Multiple simultaneous client sessions exercised during manual functional testing.

13.1 Functional Test Matrix

Test Case	Expected Result	Result
Valid login	Session established, dashboard opens	Pass
Broadcast from one client	Delivered to all other connected clients with [BROADCAST] prefix	Pass
Private message	Delivered only to the named recipient; not visible to a third connected client	Pass
Group creation and join	Group message delivered only to members who joined; non-members do not receive it	Pass
/list command	Returns current online usernames to the requesting client only	Pass
Chat history search and filter	Search text and message-type filter narrow the displayed rows correctly	Pass
Unexpected client disconnect mid-broadcast	Server removes the dead client via cleanup_client() without disrupting delivery to remaining clients	Pass

13.2 Security Test Matrix

Test Case	Expected Result	Result
Invalid password	Login rejected, failure logged	Pass
Duplicate login (same account, second client)	Second attempt rejected with DUPLICATE_LOGIN	Pass
Five consecutive failed logins	Account locked for LOCK_TIME; sixth attempt rejected immediately	Pass
Login attempt during active lockout	Rejected without re-checking the password	Pass
Login after lockout window elapses	Succeeds normally; failure counter reset	Pass
Idle session beyond SESSION_TIMEOUT	Server sends SESSION_TIMEOUT; client returns to login screen	Pass
Security log content	Every auth event recorded; no password value ever written	Pass

13.3 Testing Gaps

No automated unit or integration test suite exists for either server.py or the client modules; all evidence in Sections 13.1 and 13.2 was gathered through manual, multi-instance interactive testing and corroborated after the fact against chat_history.csv and security_log.txt. There is no fuzz or malformed-input testing beyond the basic empty-field and unrecognized-command cases described in Section 6.7, and no load testing beyond the 2-4 client range described in Section 11. These are listed as concrete recommendations in Section 15.

14. Known Limitations and Engineering Risk Register

This section consolidates every limitation, inconsistency, and defect identified during the source-code review that produced this report. It is presented as a risk register — impact and recommended action alongside each item — rather than scattered as caveats through the preceding sections, so that a reviewer can assess the system's remaining engineering debt in one place.

#	Item	Impact	Recommended Action
1	No transport-layer encryption; credentials and messages sent as plaintext TCP	High — confidentiality of all data in transit, including login credentials, is not protected	Wrap the socket with Python's ssl module (TLS); highest-priority item, see Section 15
2	Passwords hashed with unsalted SHA-256, not a password-hashing function	Medium-High — vulnerable to precomputed dictionary attack if users.json is disclosed; identical passwords produce identical hashes	Migrate to bcrypt or Argon2 with per-user salt
3	Client-side lockout countdown (30s) does not match server-side LOCK_TIME (300s)	Medium — misleads the user about when a retry will actually succeed	Read LOCK_TIME from config.json (or a value the server communicates) and drive the countdown from it
4	Server Status window's CPU/MEM and connected-user figures are randomly generated, not measured	Medium — presents simulated data as if it were live system telemetry	Integrate psutil for real CPU/memory sampling; report connected-client count from the server's actual client list
5	graph.py's message-type distribution chart uses a hardcoded array, not computed data	Medium — chart does not reflect real chat_history.csv contents	Compute counts directly from chat_history.csv before charting
6	Authentication runs synchronously in the connection-accept loop	Medium — a slow or stalled login blocks acceptance of all other incoming connections	Move authentication into the per-client thread, or accept the connection before authenticating
7	Two broadcast implementations exist (broadcast(), broadcast_all()); only one is reachable	Low-Medium — dead code increases maintenance confusion	Remove the unused broadcast() function or document its intended alternate use
8	Vestigial, disabled per-window _receive_loop() methods remain in private_chat.py and group_chat.py	Low-Medium — risk of accidental reintroduction of a socket race condition if re-enabled	Delete the dead methods; the dashboard dispatcher is the sole receiver by design
9	security_log.txt contains a duplicated line per successful login and one case-inconsistent username entry	Low — cosmetic/log-quality issue, does not affect authentication correctness	Locate and remove the duplicate security_log() call; normalize username casing at login
10	chat_history.csv contains at least one malformed row with fewer than five fields	Low-Medium — a strict CSV consumer could mis-parse or skip the row	Add field-count validation in log_chat() and defensive handling in history.py's reader

11	All demonstration accounts in users.json share an identical password hash	Low (demo-only) — should not be mistaken for five independently verified credentials	State explicitly in any external test report; use distinct passwords for production-representative testing
12	Group membership is tracked client-side only; no server query to list existing/joined groups	Low — usability limitation, not a correctness or security issue	Add a /groups command returning the server's known group list
13	Thread-per-connection server model does not scale to very large concurrent-client counts	Low at current scale (MAX_CLIENTS = 20); would become Medium-High at larger scale	Consider asyncio or a bounded worker-pool model if scale requirements grow
14	No automated test suite; all testing is manual and interactive	Medium — regressions are only caught by manual re-testing	Introduce unit tests for server routing functions and integration tests using a scripted mock client
15	Message length is not capped server-side (only a client-side UI courtesy limit exists)	Low-Medium — a malicious or malfunctioning client could send an oversized payload	Enforce a maximum message length server-side and reject or truncate oversized payloads

15. Future Enhancements Roadmap

The items below are organized by priority, derived directly from the risk register in Section 14. They represent the engineering work required to move SentinelChat from an internship-grade demonstration to a hardened, production-representative system.

15.1 Priority 1 — Transport Security

Wrap both the server's listening socket and the client's connecting socket with Python's ssl module (TLS 1.2 or later), using a self-signed certificate for development and a CA-issued certificate for any real deployment. This single change eliminates the plaintext-credential and plaintext-message exposure documented in Sections 6.8 and 12.2, and requires no change to the application-layer command protocol described in Section 5.3.

15.2 Priority 2 — Credential Hardening

Replace unsalted SHA-256 password storage with bcrypt or Argon2, each of which incorporates a per-user salt and a tunable work factor by design. This is a contained change limited to authenticate_user() and the credential-provisioning process for users.json.

15.3 Priority 3 — Genuine System Telemetry

Replace the simulated values in status.py with real measurements: psutil.cpu_percent() and psutil.virtual_memory() for CPU/memory, and a server-reported connected-client count (e.g., via a lightweight /stats command) rather than a client-side random number. This converts the Server Status window from a visual mockup into an operationally useful monitoring tool.

15.4 Priority 4 — Rigorous Performance Benchmarking

Design and execute a repeatable benchmark: fixed message size and send rate, multiple trials per client count, a documented test duration, and — specifically — a genuine before/after comparison bracketing the Assignment 8 configuration and cleanup changes, so that any future optimization claim is supported by paired data rather than narrative alone. Extend the tested client-count range beyond 2-4 to better characterize scaling behavior.

15.5 Priority 5 — Protocol and Data Robustness

- Enforce a server-side maximum message length.
- Add field-count validation when writing and reading chat_history.csv.
- Add a /groups command so clients can query server-known groups instead of relying on client-local state.
- Correct the client-side lockout countdown to read the authoritative value from config.json.

15.6 Priority 6 — Code Hygiene

- Remove the unused broadcast() function or repurpose it with a clear, documented call site.
- Delete the disabled per-window _receive_loop() methods in private_chat.py and group_chat.py.
- Deduplicate the security_log.txt double-write and normalize username casing at the point of login.

15.7 Priority 7 — Automated Testing and Scale

Introduce a unit-test suite around the server's routing functions (private_message(), group_message(), broadcast_all()) using a mock socket, and a scripted multi-client integration test that exercises the full login-through-disconnect lifecycle without manual interaction. If concurrent-client requirements are expected to exceed the current MAX_CLIENTS = 20 design point by a significant margin, evaluate migrating the server to Python's asyncio for I/O-bound scalability beyond what a thread-per-connection model comfortably supports.

16. Conclusion

SentinelChat, as implemented today, is a working, multi-client, TCP-based chat platform that successfully demonstrates the full arc of its internship objectives: reliable multi-client communication, a responsive graphical client cleanly separated from its networking layer, an application-layer authentication and session-security subsystem, and a configuration-driven, resource-cleanup-aware server. The single-receive-thread dispatcher architecture (Section 8.2) and the centralized cleanup_client() resource-management pattern (Section 10.1) are the two design decisions most worth highlighting to an engineering review board: both solve real correctness problems — message-delivery races and resource leaks, respectively — with a small, well-contained mechanism rather than ad hoc patching.

At the same time, this report has been deliberately explicit about where the system falls short of production readiness: plaintext transport, unsalted password hashing, simulated rather than measured system telemetry in the status dashboard, and a performance dataset too small to support strong quantitative claims. None of these limitations undermine the validity of the architecture or the correctness of the features that were tested (Section 13); they define the scope of engineering work required before SentinelChat could be considered for any deployment beyond a controlled demonstration or academic evaluation environment, and that work is laid out concretely in Section 15.

This document supersedes the Assignment 4, 6, 7, and 8 reports in their entirety. Any future revision of SentinelChat's documentation should extend this report rather than reintroduce a per-assignment reporting structure.

Appendix A — File and Module Reference

This appendix lists every source file reviewed for this report, its role, and the section of this document where it is discussed in detail.

File	Role	Discussed In
server.py	TCP server: accept loop, authentication, session/lockout enforcement, message routing, CSV and security logging	Sections 5, 6, 8.1, 10
client.py	Legacy terminal client (command-line, no GUI); retained as a lightweight test harness for the server	Section 4.3.3
client_gui.py	GUI login window: server IP / username / password entry, connection retry, hand-off to Dashboard	Section 7.2
dashboard.py	Central post-login window; sole owner of the socket receive thread; message dispatcher; feature-card navigation	Sections 7.3, 8.2
broadcast.py	Broadcast composer window	Section 7.4.1
private_chat.py	Private one-to-one chat window	Section 7.4.2
group_chat.py	Group chat window	Section 7.4.3
history.py	Chat history search/filter viewer reading chat_history.csv	Section 7.4.4
status.py	Server status / analytics window	Section 7.4.5
cyber_bg.py	Animated particle background canvas	Section 7.5
ui_fx.py	Shared UI helpers: GlowPulse, chat bubble rendering, system messages	Section 7.5
config.json	Externalized runtime configuration	Section 9.1
users.json	Credential store (SHA-256 hashes)	Section 9.2
chat_history.csv	Persistent message log	Section 9.3
security_log.txt	Persistent security event log	Section 9.4
performance_results.csv	Performance sample data	Section 9.5, 11.2
graph.py	Generates performance and message-distribution charts from CSV data	Section 11.1, 11.4

Appendix B — Command Protocol Quick Reference

Command	Direction	Description
LOGIN|<user>|<pass>	Client -> Server	Authentication handshake, sent once immediately after connecting
/msg <user> <text>	Client -> Server	Send a private message to a specific online user
/all <text>	Client -> Server	Broadcast a message to every connected client
/list	Client -> Server	Request the current list of online usernames
/group create <name>	Client -> Server	Create a new, empty named group
/group join <name>	Client -> Server	Join an existing named group
/group <name> <text>	Client -> Server	Send a message to all members of a named group
[PRIVATE] <sender>: <text>	Server -> Client	Delivery of a private message
[BROADCAST] <sender>: <text>	Server -> Client	Delivery of a broadcast message
[GROUP <name>] <sender>: <text>	Server -> Client	Delivery of a group message
Online Users:\n<list>	Server -> Client	Response to /list
LOGIN_SUCCESS / LOGIN_FAILED / DUPLICATE_LOGIN / ACCOUNT_LOCKED	Server -> Client	Authentication outcome
SESSION_TIMEOUT	Server -> Client	Sent when a session is closed due to inactivity

Appendix C — Glossary

Term	Definition
Dispatcher	The dashboard.py component responsible for inspecting an incoming message's prefix and routing it to the correct open feature window.
Handler thread	A per-client daemon thread on the server (handle_client()) that owns the request loop for one connection.
Lockout window	The configured duration (LOCK_TIME) during which a username is rejected at login after exceeding MAX_LOGIN_ATTEMPTS.
Session	The period between a successful login and either deliberate disconnect, idle timeout, or socket failure.
cleanup_client()	The server function that removes a client from all in-memory tracking structures and closes its socket.
Single-receiver model	The client architecture in which exactly one thread (owned by dashboard.py) ever calls recv() on the client socket.

Appendix D — References

- Python Software Foundation. Python 3 Documentation — socket, threading, hashlib, json, csv modules. https://docs.python.org
- Python Software Foundation. Tkinter — Python interface to Tcl/Tk. https://docs.python.org/3/library/tkinter.html
- Wireshark Foundation. Wireshark User's Guide. https://www.wireshark.org/docs/
- Internal project artifacts reviewed: server.py, client.py, client_gui.py, dashboard.py, broadcast.py, private_chat.py, group_chat.py, history.py, status.py, cyber_bg.py, ui_fx.py, config.json, users.json, chat_history.csv, security_log.txt, performance_results.csv, graph.py, and the project README.
- ISEA Summer Internship 2026 programme materials, Tezpur University.